Lecture 1

Introduction to Algorithmic Thinking

This lecture introduces the foundational concepts of algorithms using a no-code approach, designed for students from any educational background. The course focuses on understanding the logic and building blocks of algorithms that power modern business applications, rather than teaching programming.

Understanding Algorithms Through Everyday Examples

Tea Making Exercise

- Algorithms can be understood through familiar, real-world tasks like making tea
- An algorithm is a series of organized steps to convert input data into a desired output
- The sequence of steps may vary based on personal preference, but the final outcome remains consistent
- The process demonstrates how algorithms function in everyday situations

Core Algorithmic Terminology

Data

- Represents the actual values you work with during algorithm execution
- Changes with each iteration of the algorithm depending on the specific situation
- Example: The amount of water and milk needed varies based on the number of

people being served

Variables

- Constructs that hold data during computation
- Similar to physical vessels (saucepan, teacup) in the tea-making example
- Used to store values that may change throughout algorithm execution

Algorithm

- A series of organized steps to complete a specific task
- Steps may vary in sequence, but all work together to achieve the desired output
- Becomes increasingly complex as more details, checks, and conditions are added

Functions

- Subgroups of organized steps that are part of a larger algorithm
- Represent frequently used tasks with consistent steps across multiple algorithms
- Example: Adding numbers, calculating averages, or providing heat in cooking
- Remain the same regardless of where they appear in different algorithms

Flowchart Representation

Basic Flowchart Symbols

- Rounded Rectangle: Start and end points
- Rectangle: Processing statements and steps
- Trapezoid/Slanted Rectangle: Input and output operations
- Arrow: Flow of logic and data
- Diamond: Decision-making situations requiring branching logic

Creating Accurate Flowcharts

- Simple flowcharts contain only processing steps
- More refined versions differentiate between input, processing, and output operations
- Complex flowcharts incorporate decision loops where the algorithm revisits steps based on conditions
- Example: Tea-making flowchart with decision loop checking if tea is sufficiently brewed

Real-World Business Application: ATM Cash Dispensing

Algorithm Steps

1. Check and read card details (input)
2. Request PIN entry (input)
3. Verify PIN against stored data (decision point)

4. If PIN incorrect: end process
5. If PIN correct: request withdrawal amount (input)
6. Check account balance (decision point)
7. If insufficient balance: end process
8. If sufficient balance: release cash (function)
9. End process

Key Observations

- Not all paths in an algorithm are executed in a single run
- Depending on decision outcomes, certain paths are taken while others are skipped
- Complex operations like "release cash" are handled as separate functions with their own algorithms

Algorithm Complexity and Evolution

Simplification and Detail Addition

- All algorithms are simplifications of real-world processes
- Algorithms begin simply and become more complex as details are added
- Real-life decisions often involve intuitive steps that are difficult to codify completely
- Example: "Add water" implicitly includes finding the water, locating the container, and filling it

Deterministic vs. Complex Systems

- Modern business systems are generally deterministic and calculation-based, making them easier to algorithmize
- Physical systems (robotics, autonomous vehicles) are more complex due to real-time decision-making requirements
- Business applications require fewer considerations about physical movement

Continuous Evolution

- Algorithms and software applications are regularly updated with new features and enhanced security checks
- Each update adds new functions, additional checks, and refined conditions
- This is why quality applications receive frequent updates

Key Takeaways

- Algorithms are series of steps designed to complete specific tasks
- Both plain English and flowchart representations effectively communicate algorithmic logic
- Algorithm complexity increases with added details, conditions, and checks
- In any single execution, an algorithm

typically follows only one path based on decision outcomes

- Understanding algorithms requires recognizing how they function in both everyday situations and business applications
- Algorithms form the foundation of modern software and business systems

Lecture 2

Introduction to Pseudo Code Representation

This lecture focuses on the use of pseudo code as a method to represent algorithms in a way that bridges plain English and programming languages.

Overview

Pseudo code is a code-like representation of algorithms that is easily understandable by

programmers and developers without being tied to any specific programming language syntax. This approach allows developers to convert pseudo code into any programming language (C, C++, Java, Python, etc.) and enables people without programming experience to understand algorithmic logic through plain English.

Key Concepts and Definitions

Algorithm Components (Review)

- Data: Actual values you work with
- Variables: Constructs where data is stored during computation
- Algorithms: Organized steps to convert data into desired outputs
- Functions: Subgroups of organized steps within an algorithm

Pseudo Code vs. Code

- Pseudo Code: Plain English written in a structured, code-like manner that programmers can understand without language-specific syntax constraints
- Code: Algorithm implemented using a specific programming language with its own syntax and keyword requirements
- The distinction is important because each programming language has unique syntax rules and requirements that don't apply to pseudo code

Data Types in Algorithm Design

When defining variables, you must specify the data type to indicate what kind of values the variable will hold:

- Integer: Stores whole numbers (e.g., 60, 65, 70)

- Float: Stores decimal numbers (e.g., 62.5, 32.3)
- Boolean: Stores only two values (0 or 1, yes or no)
- Character: Stores text and letters (e.g., names, subject names)

Data Type Selection Example

- Subject marks (whole numbers only) → Integer
- Temperature measurements (can include decimals) → Float
- Name of person → Character
- Course completion status (yes/no) → Boolean

Practical Example: Calculating Average

Mathematical Approach

- Sum all marks: 60 + 65 + 70 + 55 + 60 = 310
- Divide by number of subjects: 310 ÷ 5 = 62 (average)

Algorithmic Approach (without simplification)

1. Define variables S1, S2, S3, S4, S5 as integer (for storing marks of each subject)
2. Input marks for each subject
3. Define variable "sum" as integer
4. Calculate sum = S1 + S2 + S3 + S4 + S5
5. Define variable "average" (AVG) as float
6. Calculate AVG = sum ÷ 5
7. Output the average

Simplified Algorithmic Approach (with variable consolidation)

1. Define S1 to S5 as integer; define sum as integer; define AVG as float
2. Input S1 to S5

3. Calculate sum = S1 + S2 + S3 + S4 + S5 (result: 310)
4. Calculate AVG = sum ÷ 5 (result: 62.0)
5. Output AVG

Introduction to Functions

A function is a reusable subgroup of organized steps that performs a common task. Functions can significantly reduce algorithm complexity by condensing multiple steps into a single line.

Benefits of Functions

- Simplifies algorithm representation
- Reduces lines of code
- Pre-built functions are available in programming language libraries
- Commonly available functions include: sum, average, subtraction, etc.

Function Notation in Pseudo Code

- Represented in flowcharts as rectangles with double vertical lines
- Format: variable_name = function_name(input_parameters)
- Example: AVG = average(S1, S2, S3, S4, S5)
- The function receives input values, performs calculations, and returns (stores) the result in a variable

Example with Function

```
Define S1 to S5 as integer
Define AVG as float
Input S1 to S5
AVG = average(S1, S2, S3, S4, S5)
If AVG > 60, output "First Class"
End
```

Decision Structures in Pseudo Code

Conditional statements use the "if" keyword to check conditions and execute specific actions:

- If AVG > 60: Output "First Class"
- If AVG ≤ 60: Process ends (no output, or continue to next condition)

This structure allows algorithms to make decisions based on variable values.

Flowchart Representation

The complete algorithm can be represented as a flowchart:

1. Start
2. Define variables (S1-S5, AVG)
3. Input data (S1-S5) - shown as trapezoid
4. Call function to calculate average
5. Decision diamond: Is AVG > 60?
    - Yes: Output "First Class"
    - No: End

6. End

Key Takeaways

- **Multiple Representations**: Algorithms can be represented as flowcharts or pseudo code
- **Language Independence**: Pseudo code is not tied to any specific programming language, making it universal
- **Data Type Importance**: Pre-defining data types for variables is a best practice for understanding operations and ensuring proper data storage
- **Functions as Tools**: Functions are powerful tools for simplifying algorithms by encapsulating common, reusable tasks
- **Flexibility in Learning**: Pseudo code accommodates learners with varying programming experience—those without coding background can understand through

English, while experienced programmers can quickly relate it to their preferred programming language

Lecture 3

Introduction to Algorithms and Course Overview

This lecture provided a comprehensive introduction to algorithms, data structures, and the course structure, with emphasis on understanding logic over programming implementation.

Course Information

Instructor Background

- Associate Professor in Information Systems and Digital Transformation
- Background in IT from Guru Ghasidas Central University

- MTECH in Industrial Engineering and Management from a digital perspective
- PhD in Business Information Systems from Trinity College, Dublin
- Research and teaching experience across multiple institutions including CSIR and industry roles

Course Approach

- No-code approach focusing on logic rather than programming language syntax
- Business-relevant applications of data structures and algorithms
- Reference textbook: "Codeless Data Structures and Algorithm" by Subramanya
- Complements coding courses (such as Python) by focusing on conceptual understanding

Course Objectives

- Explain common data structures and their business applications using a no-code approach
- Develop ability to identify suitable data structures for given business problems
- Compare algorithmic approaches from managerial and performance perspectives
- Apply structured logical thinking to real-world business scenarios

Course Assessment

- Quizzes: 40% (three MCQ-based quizzes; best two marks counted)
- Major Exam: 60% (descriptive, offline, taken from examination center)
- Students must attend minimum two out of three quizzes
- Quizzes can be taken online from home; major exam requires physical attendance

Course Structure

- Live sessions held on alternate weekends at scheduled times
- TA doubt-clearing sessions on alternate weekends (opposite schedule)
- Video recordings available for review
- Additional concepts and examples covered in live sessions beyond video content

Understanding Algorithms

Definition and Importance

- Algorithm: A set of organized, step-by-step instructions to convert input data into desired output
- Essential for automating tasks and enabling computers to perform complex operations
- Requires precise, detailed instructions as computers cannot assume or infer

information

Real-World Example: Making Tea

- Illustrates how intuitive processes become complex when written step-by-step
- Demonstrates need for precision when teaching machines (robots or algorithms)
- Shows importance of defining prerequisites (equipment needed) before describing processes
- Highlights necessity of adding details progressively to improve algorithm quality

Algorithm vs. Code

- Algorithm focuses on logic and problem-solving approach
- Code is the programming language implementation of an algorithm
- Same algorithm can be implemented differently across programming languages

- Algorithm quality is language-independent

Challenges in Algorithm Design

- Subjectivity in everyday language (e.g., "gentle boil," "looks okay") becomes problematic for machines
- Computers require objective, measurable criteria for decision-making
- Human interpretation and subjective judgment are areas where AI still struggles compared to human intelligence

Key Concepts

Data, Variables, and Functions

- Data: Actual values used during algorithm execution; changes with each run
- Variables: Storage locations (containers) where data is kept during computation

- Function: Reusable subgroup of organized steps that performs specific tasks
  - Functions enable code reuse across multiple programs
  - Can be called multiple times within a single algorithm
  - Simplify complex algorithms by encapsulating common operations

Why Variables Matter

- Enable algorithm reusability across different datasets
- Allow same algorithm to work with different input values
- Example: Average calculation function works with any set of numbers, not hardcoded values

Data Types

Purpose of Data Types

- Define what kind of data a variable can store
- Optimize memory usage (smaller data types for specific needs)
- Prevent errors by restricting invalid inputs
- Classical languages require predefined data types; modern languages (Python) can infer types

Common Data Types

Data Type	Purpose	Example
Integer	Whole numbers only	Marks (0-100), number of students
Float/ Floating Point	Decimal numbers	Temperature, interest

		rates, CGPA
Boolean	Two values only (true/ false or 0/1)	Eligible/not eligible, pass/ fail
Character/ Text	Text and strings	Names, addresses, book titles

Data Type Selection Considerations

- Business domain understanding required
- Determined through consultation with end-users and domain experts
- Role of business analysts: bridge between business requirements and technical implementation
- Example: Customer ID with letters and numbers must be stored as text, not pure number

Algorithm Representation Methods

Flowchart Notation

- Rounded Rectangle: Start and end points
- Rectangle: Standard processing steps
- Trapezoid: Input/output operations
- Diamond: Decision points (yes/no branching)
- Arrows: Show flow direction and logical sequence

Limitations of Flowcharts

- Become complex and difficult to follow with multiple conditions and loops
- Can be simplified using function notation
- Additional diagramming techniques (Data Flow Diagrams, UML) help with complex systems

Pseudocode

- English-language representation of algorithm steps
- Bridge between flowchart and actual programming code
- Easier to convert to programming code than flowcharts
- More structured than plain English, more readable than code
- Allows language-agnostic algorithm description

Practical Application: Average Calculation

Variable Definition Approach

```
Define S1, S2, S3, S4, S5 as integers
Input S1, S2, S3, S4, S5
Define Sum as integer
Sum = S1 + S2 + S3 + S4 + S5
```

```
Define Average as float
Average = Sum / 5
Output Average
```

Why Use Variables Instead of Hardcoding

- Hardcoded values (e.g., 62 + 65 + 70...) make function non-reusable
- Variables enable same algorithm to work with different datasets
- Each execution uses different data, not different code

Decision-Making (If Statements)

- Implement diamond shapes from flowcharts
- Allow conditional outputs based on calculated results
- Example: If average > 60, output "First Class"
- Multiple conditions handled through sequential if-else statements

Functions

Function Representation in Pseudocode

- Function name followed by parentheses containing parameters
- Example: Average(S1, S2, S3, S4, S5)
- Separates common, reusable operations from main algorithm
- Shows input parameters and produces specific output

Benefits of Function-Based Design

- Reduces algorithm complexity by delegating tasks
- Enables code reuse across multiple programs
- Makes algorithms more maintainable and easier to modify
- Many standard functions available in

programming libraries

When to Use Functions

- Operations needed multiple times in program
- Common calculations (sum, average, sorting)
- Standard operations available in programming libraries (especially Python)

Business Context and Real-World Considerations

ATM Transaction Example

- Demonstrates practical algorithm with multiple decision points
- Shows importance of error handling and validation
- Illustrates sequential decision-making (card validity → PIN verification → balance check)
- Highlights real-world complexities

Data Entry Validation

- System-level checks validate data type and range (e.g., marks between 0-100)
- Human/managerial perspective identifies logical errors (wrong data entry)
- Example: System accepts 63 as valid mark, but employee may have intended 65

Business Analyst Role

- Collects requirements from business domain experts and end-users
- Prepares Software Requirement Specification (SRS) document
- Converts business needs into technical requirements for developers
- Acts as bridge between business and technology teams
- Creates design documents and diagrams for

technical implementation

Real-Time Data Processing Decisions

- Not all functions need real-time execution
- Example: UPI transfers show immediate user confirmation but backend settlement happens periodically (every two hours)
- Daily summary functions (total sales) often run at fixed intervals (midnight) rather than continuously
- Business requirements determine processing frequency

System Maintenance Considerations

- Periodic maintenance windows required for running heavy computations
- Backend processes may differ from user-facing operations
- Resource optimization balances responsiveness with system efficiency

Important Distinctions

Algorithm vs. Programming Language

- Algorithm is independent of language choice
- Same algorithm may be easier or harder to implement in different languages
- Algorithm focuses on "what to do"; code focuses on "how to implement"

One-Time vs. Reusable Solutions

- Legacy approach: Custom programs written for individual companies
- Modern approach: Cloud-based solutions reuse same code for multiple users/ organizations
- Parameterization enables single codebase to serve diverse needs

Pseudocode vs. Programming Code

- Pseudocode uses English-like language
- Programming code uses language-specific syntax
- Pseudocode is intermediate step facilitating conversion to actual code

Student Misconceptions Addressed

- Algorithms cannot be trained by computers initially; computers operate through pre-existing algorithms
- Precise descriptions do not guarantee perfect results if precision definition is ambiguous
- Algorithms and coding go hand-in-hand but are conceptually distinct
- Functions are reusable components of algorithms, not replacements for algorithms
- Boolean data type preferred over integer for true/false values due to memory efficiency

and error prevention
- Flow diagrams must use sequential diamonds for multiple conditions; single diamond allows only yes/no branching

Course Continuation Structure

- Lecture today focuses on fundamental concepts and terminology
- Previous video sessions covered similar content with additional examples
- Encourages watching video content during week for better understanding
- Live sessions serve as recapitulation with additional interactive elements and clarifications
- TA sessions reserved for doubt resolution on alternate weekends

Lecture 4

Properties of Algorithms in Real-World Business Contexts

This lecture covers the essential properties that define a good algorithm, with emphasis on how these properties apply to practical business situations. The course uses a business-focused approach rather than a purely computational one to demonstrate how algorithms function in real-world settings.

Overview of Algorithm Properties

Algorithm characteristics are divided into two main categories:

Core Technical Characteristics

These properties focus on the fundamental nature of the algorithm itself, regardless of

implementation method (plain English, pseudocode, or flowchart):

- **Finiteness**: Algorithm must terminate after a countable, finite number of steps
- **Definiteness**: Each step must be clearly defined with no ambiguity
- **Well-Defined Inputs**: External data requirements must be explicitly specified
- **Well-Defined Outputs**: Algorithm must produce at least one clear expected result
- **Effectiveness**: Algorithm must solve the problem efficiently and usefully

Execution and Design Characteristics

These properties determine whether an algorithm can be properly implemented and generalized:

- **Language Independence**: Algorithm works

regardless of programming language
- **Determinism**: Same input produces same output consistently
- **Feasibility**: Algorithm can actually be implemented
- **Generality**: Algorithm applies broadly to similar problems

Detailed Property Analysis

1. Finiteness

Definition: An algorithm must end after a countable, finite number of steps without looping indefinitely.

Valid Example - Classroom Student Count:

- Start with first student at first desk
- Count each student exactly once
- Move to next desk progressively
- Stop at last student and display total

- The algorithm terminates with a finite, countable number of steps (e.g., 40 or 600 students)

Invalid Example - Retail Store Customer Counter:

- Check if customer has entered
- If no, check again (return to step 1)
- If yes, count and return to step 1
- This creates an infinite loop because the system continuously checks for incoming customers with no defined endpoint

Solution: Add a reset condition (e.g., "reset counter to zero at midnight") to make the algorithm finite and collect meaningful data within defined time periods.

2. Definiteness and Unambiguity

Definition: Each step must have exactly one clear interpretation with no room for multiple

interpretations.

Valid Example - Calculate Average of Marks:

- Input five marks (M1, M2, M3, M4, M5)
- Calculate average: (M1 + M2 + M3 + M4 + M5) ÷ 5
- Display average
- Stop
- Each step is precise and unambiguous with clear numerical operations

Invalid Example - Ambiguous Tea-Making Algorithm:

- Take some tea leaves
- Add a little milk
- Boil for a while
- Add sugar if needed
- Problems: "some," "little," and "a while" are subjective interpretations
- Different people will produce different results

- First-time learners cannot follow this without outside knowledge

Key Insight: Good recipes and algorithms specify exact measurements (e.g., "250 ml of milk" instead of "a little milk"). Machines and computers cannot make subjective interpretations—they require precise, quantified instructions.

Connection to Artificial General Intelligence: The challenge of creating AGI stems partly from this property. While humans easily interpret ambiguous instructions like "come near me," machines require explicit quantitative specifications. Current AI systems work well with specific, definite tasks but struggle with ambiguous, general-purpose instructions.

3. Well-Defined Inputs

Definition: External data requirements must be

explicitly and clearly specified.

Valid Example - Rectangle Area Calculation:

- Input: Length (L) and Width (W)
- Calculate: Area = L × W
- Display area
- Stop
- Inputs are specific: exactly two values (length and width) are required

Invalid Example - Vague Input Specification:

- Input the required value
- Calculate the area
- Problems: "required value" is unclear; doesn't specify what values are needed
- "Calculate the area" doesn't indicate which formula or inputs to use
- Both input specification and the step itself are ambiguous

4. Well-Defined Outputs

Definition: Algorithm must produce at least one clear, expected result that matches problem requirements, even when conditions are not met.

Valid Example - Loan Approval Process:

- Input: Monthly income and credit score
- Apply criteria: Income > 50,000 AND Credit score > 600
- Output: "Loan Approved" OR "Loan Rejected"
- The algorithm always provides a clear, definitive output
- User knows the decision regardless of outcome

Invalid Example - Undefined Output:

- Input applicants' details (vague input)
- Apply loan approval criteria
- Display: "Processing Complete"

- Problems: User doesn't know if application is approved or rejected
- "Processing Complete" provides no actionable information
- Both input and output lack clarity

Real-World Parallel: Good customer service (like email responses saying "We're working on it, will respond in 48 hours") provides well-defined output even when the complete solution isn't ready. The algorithm should always communicate a clear status or result.

5. Effectiveness

Definition: Algorithm must not only solve the problem but do so in a practical, useful manner considering time, resources, and business objectives.

Valid Example - Inventory Reordering System:

- Input: Current stock quantity

- Define: Safety stock threshold = 50 units
- Define: Purchase order quantity = 200 units
- Logic: If stock < 50, create purchase order for 200 units; otherwise, maintain current stock
- Effectiveness: Balances production continuity (doesn't risk running out) with cost management (doesn't overstock)
- Avoids both production interruption and excessive storage costs

Invalid Example - Ineffective Inventory Algorithm:

- Check current stock
- Order more items whenever stock looks low
- Problems: "Low" is undefined and subjective; different managers interpret differently
- "Order more items" doesn't specify quantity; some order 200, others order 1,000
- Results in inconsistent, costly decisions
- May cause either production risk or

excessive inventory costs
- Not effective for organizational goals

Resource and Time Considerations:

- An algorithm that solves a problem in 3 hours is ineffective for real-time loan approval
- An algorithm requiring excessive computing resources is ineffective for devices with limited capabilities
- Effectiveness encompasses not just correctness but practicality

Interconnection of Properties

All algorithm properties work together:

- An algorithm with infinite loops violates finiteness and compromises effectiveness
- Missing well-defined outputs often means

well-defined inputs are also unclear
- Violating definiteness makes it impossible to achieve well-defined outputs

Practical Exercise Examples

Example 1: Flawed Algorithm

- Step 1: Input applicant details
- Step 2: Evaluate eligibility
- Step 3: Return to Step 2
- Property Violated: Finiteness (creates infinite loop)
- Secondary Issue: Well-defined output (never reaches conclusion)

Example 2: Ambiguous Discount Algorithm

- Input: Customer purchase amount
- If purchase amount is large, give large discount

- Display final bill and stop
- **Property Violated**: Definiteness (lacks precise definitions)
- **Problem**: "Large" purchase and "large" discount are undefined
- **Impact**: Different staff apply different criteria inconsistently

Summary of Key Takeaways

- **Core technical properties** (finiteness, definiteness, inputs, outputs, effectiveness) define the algorithm itself
- **Execution properties** determine implementation feasibility and applicability
- Business effectiveness requires balancing multiple properties simultaneously
- Real-world algorithms must be both theoretically sound and practically useful
- Ambiguity and vagueness are enemies of

proper algorithms in computational contexts

Lecture 5

Second Set of Algorithm Properties and Types of Algorithms

Overview

This lecture covers the second set of desirable properties that algorithms should possess for effective implementation, along with the main types of algorithms used to solve different problems.

Execution and Design Characteristics

Language Independence

- A good algorithm should be expressed in

logical steps that are generic and not tied to any specific programming language

- The algorithm's logic must be understandable regardless of the programming language used for implementation
- Example: A bonus calculation algorithm expressed in plain English is language-independent, while the same logic written in Python-specific syntax is language-dependent
- Language-independent algorithms can be translated into any programming language while maintaining the same logic

Determinism

- An algorithm is deterministic when the same input consistently produces the same output across multiple executions

- This property ensures predictability and reliability in algorithmic behavior
- Example of Deterministic Algorithm:
  - Loan approval process: Fixed criteria (monthly income > 50,000 rupees AND credit score > 700) always produces the same approval/rejection decision
  - The same set of inputs will always result in the same output
- Example of Non-Deterministic Process:
  - Visa approval: Depends on visa officer's subjective judgment about trustworthiness, which can vary between officers and interviews
  - Same application details may result in different outcomes depending on multiple factors beyond fixed criteria
- Algorithms must be deterministic to be reliably implementable in computing systems

Feasibility

- An algorithm must be **implementable realistically within available resources** such as system memory, processing power, and time constraints
- Historical examples of feasibility challenges:
  - **Cloud Computing:** Technology existed in the 1990s but lacked feasibility due to slow internet speeds and high costs; became popular in recent years with improved internet speeds and cheaper infrastructure
  - **Machine Learning/AI:** Algorithms existed since the 1970s but became feasible only recently due to cheaper computing power, increased data availability, and better memory capacity
- **Payroll Processing Example:**
  - **Feasible Approach:** Read employee salary

    - records one by one from the database, calculate net salary, generate pay slip, repeat for next employee
    - **Infeasible Approach:** Load all employee records into memory simultaneously and process all at once, which is very costly and impractical for regular computing systems
- An algorithm must solve the problem within time and resource constraints; otherwise, organizations may not invest in its implementation

Generality

- An algorithm should be **general enough to handle all valid instances** of a problem without special cases
- The algorithm must work for all valid inputs, not just specific or unique cases

- Example of Lack of Generality:
  - Assigning bonus only to employees named "Rahul" lacks generality and is specific to one case
  - This approach is unethical and not implementable in business
- Valid vs. Invalid Inputs:
  - Invalid inputs (negative sales values, letters instead of numbers) may fall outside the algorithm's scope
  - Algorithms should handle all valid inputs consistently without special case conditions
- A good algorithm applies universally to the problem domain, not to particular individuals or scenarios

Application of Properties: Review Questions

Question 1: Discount Algorithm Analysis

- **Algorithm:** Randomly chooses a discount between 5% and 20% for all customers
- **Property Not Satisfied:** Determinism
- **Reason:** Random selection introduces uncertainty; the same purchase amount may result in different discount percentages each time, violating the principle that same inputs should produce same outputs
- While randomness may be part of company policy, it goes against algorithmic principles

Question 2: Customer Complaint Algorithm Analysis

- **Algorithm:** Customer submits complaint → Determine customer's true intention → Provide solution
- **Property Not Satisfied:** Feasibility

- Reason: The second step (determining customer's true intention) is not strictly executable by a computing system without extremely complex solutions (like AI with facial recognition), making it impractical for a standard complaint handling system

Types of Algorithms

1. Divide and Conquer Algorithm

- Core Concept: Divide a large problem into smaller sub-problems, solve each independently, and combine results to obtain the final solution
- Everyday Analogy: Group projects where team members work on different sections and combine work at the end
- Business Example - Company Audit:
  - Large company divides audit work among

regional offices (North, South, East, West, Northeast)
    - Each regional team audits their respective office accounts
    - All regional audit reports are combined into a single company-wide audit report
    - Smaller companies may use a single auditor who examines each office sequentially and combines findings

2. Greedy Algorithm

- **Core Concept:** Makes the best possible decision at the current point in time based on available information and constraints
- **Key Characteristic:** Assumes that taking the locally optimal choice will lead to a useful overall solution
- **Decision-Making Approach:** Takes the best available option now, not necessarily the

globally optimal solution

- Business Example - Making Change:
  - Customer pays 500 rupees for a 150 rupee ticket (change due: 350 rupees)
  - Cashier chooses one 200-rupee note, one 100-rupee note, and one 50-rupee note (greedy choice: largest denominations first)
  - This is faster and more practical than counting 35 ten-rupee notes
  - Locally optimal choice produces a practical solution quickly
- Investment Example: Investor with limited budget chooses stocks offering the highest immediate return, potentially selling later when better opportunities emerge

3. Dynamic Programming Algorithm

- Core Concept: Makes decisions for the

future based on analysis of past data and current conditions

- Key Characteristic: Uses historical data and patterns to inform future planning
- Business Example - Inventory Reordering:
  - Reviews past three years of purchase history, holding costs, and demand patterns
  - Analyzes whether future demand will increase or decrease
  - Plans inventory accordingly for the next 12 months
  - Air Cooler Manufacturing Example: Order more inventory before March (summer season when demand peaks) but reduce orders before rainy season (when cooler demand decreases and cold season approaches with potential demand increase)
- Core Principle: Look at past patterns and

current inventory to plan optimal future purchasing decisions

Algorithm Type Matching Exercise

Situation 1: Production Planning

- Scenario: Manufacturer decides monthly production by considering current inventory, previous production levels, and expected future demand
- Suitable Algorithm Type: Dynamic Programming
- Reason: Uses past production data and inventory levels to predict and plan for future demand

Situation 2: National Sales Reporting

- Scenario: National retail chain asks each regional office to prepare sales reports; head

office combines all regional reports into national report

- Suitable Algorithm Type: Divide and Conquer
- Reason: Breaks down national-level sales calculation into smaller regional problems, then combines regional solutions

Situation 3: Investment Portfolio

- Scenario: Investor with limited budget invests in opportunities offering the highest immediate return before considering others
- Suitable Algorithm Type: Greedy Algorithm
- Reason: Focuses on locally best solutions at each point in time; may sell investments later if better opportunities emerge, following immediate best returns

Summary of Algorithm Characteristics

Core Technical Characteristics (5 Properties)

- Finiteness
- Definiteness
- Well-defined inputs
- Well-defined outputs
- Effectiveness

Execution and Design Characteristics (4 Properties)

- Language independence
- Determinism
- Feasibility
- Generality

Main Algorithm Types

- Divide and Conquer
- Greedy Algorithm

- Dynamic Programming

Lecture 6

Introduction to Linear Data Structures and Arrays

This lecture introduces linear data structures, focusing on arrays as the primary example of organizing data sequentially for improved accessibility and management.

Overview of Data Concepts

- **Data**: Actual values you work with during computation
- **Variables**: Storage locations for data during program execution
- **Limitation of simple variables**: Declaring individual variables for each piece of data (e.g., sub1, sub2, sub3) becomes

cumbersome and inefficient when dealing with multiple instances of similar data types

Linear Data Structure Definition

Linear data structures arrange elements sequentially in an ordered manner, where each element is positioned relative to others. The key benefit is that if you can access one element, you can find adjacent elements through a defined relationship. This provides a structured way to organize and access data efficiently.
Real-world analogy: Comparing traditional addresses (requiring knowledge of landmarks) with modern structured addresses using house numbers and postal codes, which allow direct access regardless of ownership or location changes.

What Are Arrays?

Arrays are a fundamental linear data structure with the following characteristics:

- **Homogeneous data type:** All elements must be the same data type (all integers, all floats, or all characters—cannot mix types)
- **Indexed access:** Individual elements are identified using index numbers, enabling direct access without reading sequentially
- **Index convention:** Indexing begins at 0, so the first element is at index 0, the second at index 1, and so forth
- **Contiguous memory storage:** Elements are stored in continuous memory locations

Array Declaration Syntax

C/C++ format:

```
int sub[5] = {60, 65, 70, 50, 60};
```

(Requires explicit data type and size specification)

Python format:

```
sub = [60, 65, 70, 50, 60]
```

(Data type is inferred from content; more flexible)

Single-Dimensional Arrays

- Store elements in a single row with indices from 0 to n-1
- Example use case: Storing marks of five subjects for one student
- Direct access example: sub[3] retrieves the value at index 3

Multi-Dimensional Arrays

Two-dimensional arrays extend the concept to

handle more complex data organization:

- Structure: Rows represent one dimension (typically indexed as i), columns represent another (indexed as j)
- Use case: Storing marks for multiple subjects across multiple students
- Index notation: array[i][j] where i is the row index and j is the column index
- Example: A 7×5 array stores marks for 7 students across 5 subjects, accessed as sub[2][1] for row index 2, column index 1

Note: Arrays can theoretically have n dimensions, but two-dimensional arrays are most practical for typical applications. Higher dimensions increase programming complexity.

Basic Array Operations

1. Read/Access

- Direct access: Retrieve a value from a known index position instantly
- Sequential access: Read elements one by one, useful for aggregating data (e.g., calculating weekly sales totals)
- Performance: Instantaneous if index is known

2. Search

- Linear search: Start at index 0 and compare each element sequentially with the search term
- Process: Increment index and repeat comparison until the element is found or array ends
- Performance: Depends on element position; faster if element is near the beginning, slower if near the end

3. Insert

- At the end: Simple operation if array has available space
- At beginning or middle: Requires shifting all subsequent elements forward to create space
- Example: Inserting at index 0 requires moving element at index 4→5, 3→4, 2→3, 1→2, 0→1
- Performance: Time-consuming if insertion requires shifting multiple elements

4. Update

- Known location: Directly go to the index and replace the value; instantaneous operation
- Unknown location: Combine search with update; requires finding the element first, then updating

- Example: Updating inventory when stock is sold (15 units → 10 units after selling 5)

5. Delete

- Operation: Remove or set a value at a specific index to null (not zero, but "nothing stored")
- Known location: Direct deletion is instantaneous
- Unknown location: Requires searching first, then deleting
- Storage: The memory space remains allocated; it is not reclaimed

Performance Analysis Summary

Operation	Known Location	Unknown Location

Read/Access	Instantaneous	Sequential reading required
Search	Instantaneous	Depends on position
Insert	Easy (at end)	Time-consuming (requires shifting)
Update	Instantaneous	Depends on search location
Delete	Instantaneous	Depends on search location

Advantages of Arrays

- **Easy to understand**: Simple extension of the single-variable concept
- **Memory efficient**: Continuous storage allocation for a known fixed size
- **Fast direct access**: Instantaneous retrieval when index is known
- **Foundation for other structures**: Many advanced data structures are built upon arrays
- **Ideal for predictable datasets**: Excellent when data size is known in advance

Limitations of Arrays

- **Fixed length**: Size must be declared at creation and cannot be easily extended
- **Size prediction challenges**: Too large a declaration wastes memory; too small causes overflow
- **Resizing complexity**: Cannot simply add

elements; requires creating a new array, copying data, then inserting new values
- Contiguous memory requirement: Must use continuous memory blocks; cannot work with scattered memory locations
- Inefficient with frequent modifications: Operations combining search with update/delete consume significant time and resources
- Search performance degradation: Sequential searching becomes increasingly time-consuming as array size grows

Key Takeaways

- Linear data structures arrange elements sequentially for structured, efficient access
- Arrays store homogeneous data types in indexed, contiguous memory locations
- Single-dimensional arrays suit simple

sequential data; multi-dimensional arrays handle complex, tabular data
- Direct index-based access is highly efficient; sequential searching is resource-intensive
- Arrays excel with predictable, fixed-size datasets but struggle with dynamic data or frequent modifications
- Understanding operation performance is crucial for choosing appropriate data structures for specific use cases

Lecture 7

Introduction to Linked Lists and Linear Data Structures

This lecture continues Module 2 on linear data structures, focusing on linked lists as the second major linear data structure after arrays. The session explains why linked lists are

necessary and how they differ from arrays in terms of flexibility and use cases.

Overview of Linear Data Structures

- Linear data structures arrange elements sequentially with a logical order
- Elements are connected so that accessing one item allows movement to adjacent items
- Arrays use indexing to provide direct access to elements

Limitations of Arrays

- Fixed Size Requirement: Array size must be defined in advance; adding elements beyond the initial size requires redefining the entire array
- Inefficient Modifications: Deleting or inserting elements in the middle requires

shifting other elements or creating null spaces
- Static Structure: Limited flexibility when data requirements change
- Best Use Case: Arrays work well when you know the exact number of elements needed in advance and direct access is a priority

Need for Linked Lists

Linked lists address the limitations of arrays in scenarios where:

- The number of elements is unknown in advance
- Frequent addition, deletion, and rearrangement of items is required
- Flexibility and dynamic sizing are essential

Practical Example - Music Playlists:

- Users can add or remove songs without predefined limits
- Songs can be reordered based on preference
- New songs can be inserted at any position in the sequence

Fundamental Concepts of Linked Lists

Basic Structure:

- Each linked list starts with a **head** node (the entry point, similar to a bus stop)
- Each node contains data and a connection (pointer/address) to the next node
- The last node points to **null**, indicating the end of the list
- Elements are not stored in contiguous memory; they can be scattered anywhere in memory
- The connections between nodes create the

logical sequence, not physical proximity

Key Difference from Arrays:

- Arrays store elements in consecutive memory locations
- Linked lists use pointers/addresses to link elements regardless of their physical memory locations

Types of Linked Lists

Single Linked List:

- Nodes connect in one direction only (forward)
- Allows traversal from head to end
- Useful for sequential playback (play next functionality)

Doubly Linked List:

- Nodes have connections in both directions (forward and backward)
- Enables movement both forward and backward through the list
- Implements features like "play previous" in music players

Circular Linked List:

- The last node points back to the head instead of null
- Creates a loop that returns to the beginning when reaching the end
- Useful for implementing repeat or shuffle features
- Common in menu systems and cyclical applications

Operations on Linked Lists

Adding Elements:

- New elements can be inserted anywhere in the list
- The process involves placing the new element in any available memory location
- Pointer connections are adjusted so that the previous node points to the new node, and the new node points to the former next node
- No need to shift or reorganize existing elements

Deleting Elements:

- Removal requires only adjusting the pointer connections
- The previous node is redirected to point to the node after the deleted element
- The deleted element remains in memory but is no longer part of the active list
- Extremely efficient compared to array deletion

Reshuffling (Randomization):

- The actual song/data elements remain in their memory locations
- Only the pointer connections are rearranged to create a different order
- Allows random playback without physically moving data
- Multiple reshuffles can create different orderings of the same elements

Nested Linked Lists (Linked List of Linked Lists)

Linked lists can be combined hierarchically to handle complex data structures with multiple levels of relationships.

Use Cases:

- University semester structure with courses
- Multi-playlist systems

- Hierarchical organizational data

Comparison of Data Structures:

Scenario	Best Data Structure	Reason
Fixed number of known elements with direct access needs	Array (2D Array for multiple dimensions)	Efficient memory usage and fast direct access
Variable number of elements with frequent additions/ deletions	Linked List	Flexibility and ease of modification

Hierarchical structure with varying number of sub-elements	Nested Linked Lists	Accommodates complexity and variability at multiple levels

Practical Example - Academic Program Structure:

- Situation 1 (Fixed Structure): 8 semesters with 6 courses each → Use 2D array
- Situation 2 (Variable Structure): Varying number of semesters and courses per semester due to student circumstances, retakes, or deferrals → Use linked list of linked lists

Key Takeaways

- Flexibility: Linked lists provide dynamic sizing through pointer-based connections
- Memory Efficiency: Elements need not be stored contiguously; memory can be used anywhere
- Directional Variants: Single, doubly, and circular linked lists address different navigation requirements
- Operation Efficiency: Insertion and deletion are simpler than in arrays as no shifting is required
- Hierarchical Applications: Nested linked lists handle complex, multi-level data structures with varying sizes
- Practical Implementation: Modern programming languages provide built-in linked list implementations; understanding the conceptual mechanics is essential for effective use

Lecture 8

Understanding Algorithm Properties and Linear Data Structures

Overview

This lecture covered two main topics: the essential properties that algorithms must possess and an introduction to linear data structures, specifically arrays and linked lists. The session emphasized how algorithms should be designed with specific characteristics to ensure they work effectively in real-world applications, while data structures provide organized ways to store and manage data efficiently.

Algorithm Properties

Technical Properties

Finiteness

- An algorithm must terminate after a countable, finite number of steps
- It should not loop indefinitely
- Example: Counting students in a class will eventually end, whereas continuously waiting for new customers entering a store without a time limit would be infinite

Definiteness

- All steps must be clear and unambiguous with no confusion
- Instructions should be precise enough to be understood by anyone following them
- Example: "Calculate average by adding five numbers and dividing by five" is definite, whereas vague instructions like "add some tea powder" in a tea-making process are ambiguous

Well-Defined Input

- Input requirements must be clearly specified
- Example: For calculating rectangle area, you must specify that length and width are needed, not just "required values"

Well-Defined Output

- The expected output must be clearly stated
- Example: A loan approval algorithm should specify "Approved" or "Rejected," not just "Processing complete"

Effectiveness

- Every step must be basic enough to execute in finite time using simple operations
- Example: Inventory management algorithm stating "if stock is less than 50 units, order 200 units" is effective, whereas "check stock and order if needed" without specifying

threshold is vague

Execution and Design Properties

Language Independence

- Algorithms should not depend on any specific programming language
- They should be expressed in plain English or pseudocode understandable to anyone
- Implementation choice (C, Python, Java) comes later

Determinism

- Same input must always produce the same output
- Example: Loan approval (based on quantitative criteria) is deterministic, whereas visa application (involving subjective interviews) is not

Feasibility

- An algorithm must operate realistically within available system memory and processing power
- Must be implementable with available resources (money, time, space)
- Example: Processing all employee payroll sequentially is feasible; attempting parallel processing of simple calculations is infeasible

Generality

- Algorithm should handle all possible situations, not just specific cases
- Example: Don't hardcode employee names or fixed number of subjects; make it flexible to accept any number of subjects or employees

Algorithm Approaches

Divide and Conquer

- Break large problems into smaller subproblems, solve them separately, then combine results
- Example: Regional sales analysis (dividing by regions, analyzing each, then combining for total)

Greedy Algorithm

- Make locally optimal choices to find a quick solution within available resources
- May not be globally optimal but works well for practical purposes
- Example: Cashier returning change using largest denominations first

Dynamic Programming

- Consider past solutions and decisions when making current decisions
- Example: Production companies planning based on seasonal trends from previous quarters

Linear Data Structures

Arrays

Characteristics

- Elements stored in sequential locations with specific indexes
- Fixed size must be defined during creation
- Same data type for all elements
- Indexes typically start from 0
- Can be single or multi-dimensional

Basic Operations

- Read/Access: Retrieve values from specific index (fast if index known)
- Search: Find specific items within array (sequential search if index unknown)
- Insert: Add new elements (easiest at end, more complex in middle as items must shift)
- Update: Modify existing values (fast if index known)
- Delete: Remove content at index (only clears data, doesn't free memory)

Benefits

- Simple to understand and implement
- Quick access if index is known
- Memory efficient for known fixed-size data
- Ideal for predictable data sets

Limitations

- Fixed length; resizing is resource-intensive

- Requires contiguous memory locations
- Insertion and deletion in middle can be time-consuming
- Not suitable for frequently changing data

Linked Lists

Characteristics

- Items connected through memory references (links/pointers)
- No fixed size; can grow dynamically
- Each node contains data and reference to next node
- Starts with a "head" pointer
- Flexible arrangement of items

Types

- Singly Linked List: One-directional links (forward only)
- Doubly Linked List: Two-directional links

(forward and backward)
- Circular Linked List: Last node points back to head for continuous cycling

Basic Operations

- Add: Insert anywhere by rearranging memory references
- Remove: Delete by updating linkages
- Rearrange: Change order by modifying pointers
- Navigate: Move through items following links

Benefits

- Dynamic size; can add unlimited items
- Flexible insertion and deletion at any position
- No need for contiguous memory
- Ideal for frequently changing data

Limitations

- More complex to understand and implement
- Slower access compared to arrays (must follow links)
- Additional memory required for storing references
- Cannot randomly access items by index

Comparison: Arrays vs. Linked Lists

Arrays are preferred when data size is known and predictable, frequent access is needed, and simplicity is important. Linked lists are preferred when data size is unknown, frequent insertion/deletion occurs, and flexibility is required. A practical example: Music playlists benefit from linked lists because songs can be easily added, removed, or reordered without restructuring the entire collection.

Foundation Importance

Both arrays and linked lists serve as foundational data structures. Other data structures such as stacks, queues, trees, and graphs are typically implemented using either arrays or linked lists, making understanding these basics essential for computer science applications.

Lecture 9

Stacks: A Linear Data Structure with Last-In-First-Out Operations

Overview

Stacks are linear data structures that follow the Last-In-First-Out (LIFO) principle, where the most recently added element is the first one to be removed. This lecture covers the fundamental concepts, operations, and practical applications

of stacks, including their implementation approaches and common errors.

Real-World Applications and Examples

Undo Functionality in Word Processors:

- When using applications like Microsoft Word, the undo feature relies on stack operations
- Each change made to a document is stored in a stack
- Clicking undo removes changes in reverse chronological order (last change first)
- This behavior directly corresponds to stack's LIFO nature

Physical Stack Examples:

- Stack of plates at a buffet or wedding
- Stack of newspapers
- Weighing plates in a gymnasium

- In all cases, items are added and removed from the top only

Stack as a Data Structure

Basic Definition:

- A stack is a data structure where new elements are always stored at the top of the list
- Elements are retrieved (removed) only from the top
- Logically organized memory spaces behave in a stack manner, though physical memory locations may not be hierarchical

Key Principle:

- Whatever element was put in last remains at the top and is removed first
- This is why stacks are called LIFO (Last In

First Out) data structures

Core Stack Operations

1. Push Operation:

- Adds a new element to the top of the stack
- Requires specifying the value to be pushed
- Example: push(64) adds 64 to the stack top

2. Pop Operation:

- Removes and returns the top element from the stack
- Does not require specifying a value (removes whatever is on top)
- Example: pop() removes the current top element

3. Peak Operation:

- Examines the top element without removing

it
- Useful for checking what's available before deciding whether to pop
- Example: peak() returns the value at the top

4. Size Operation:

- Returns the total number of elements currently in the stack
- Useful for memory management and constraint checking
- Example: size() returns the count of elements

Types of Stacks

Static Stacks:

- Fixed in size and predefined during creation
- Cannot grow beyond the specified limit
- Typically implemented using arrays
- Examples:

    - Adobe Photoshop: Default limit of 50 undo operations
    - Microsoft Excel: Default limit of 100 undo operations
- Settings can be customized based on available memory and system capability

Dynamic Stacks:

- Variable in size and grow as needed
- More flexible than static stacks
- Typically implemented using linked lists
- Example: Microsoft Word uses dynamic stacks for undo operations
- Can grow until memory limitations are reached
- Practically limited by available RAM and system resources

Stack Errors

Stack Overflow:

- Occurs when attempting to push an element into a full stack
- Can happen with both static stacks (by definition) and dynamic stacks (due to memory limitations)
- Results from exceeding allocated memory or capacity
- Common error in programming (noted by the popularity of Stack Overflow website)
- Real-world example: Opening too many browser tabs causes memory overflow

Stack Underflow:

- Occurs when attempting to pop an element from an empty stack
- Happens when more pop operations are performed than available elements
- Results in a null or undefined error

- Real-world example: Clicking the browser back button when no previous pages exist

Implementation Approaches

Static Stack Implementation:

- Uses arrays as the underlying data structure
- Array size must be predefined
- Suitable when maximum capacity is known and limited
- More efficient for fixed-size requirements

Dynamic Stack Implementation:

- Uses linked lists as the underlying data structure
- Allows for flexible growth and shrinkage
- Suitable when size requirements are variable
- Better for applications with unpredictable storage needs

Advanced Concepts

Stack of Stacks:

- Concept of nested stacks where each element in a stack is itself a stack
- Real-world example: Multiple browser windows, where each window contains multiple tabs
- Each tab history could be represented as a stack within the main stack

Memory Considerations:

- Static stacks: Stack is cleared when file is saved (in applications like Excel and Photoshop)
- Dynamic stacks: Limited by physical memory (RAM) of the device
- System crashes may occur if memory is exhausted

- Frequently saving files helps manage stack size in applications

Key Takeaways

- LIFO Principle: Stacks operate on the last-in-first-out principle, making them ideal for undo operations and backtracking
- Four Main Operations: Push, pop, peak, and size cover all standard stack operations
- Implementation Choice: Static stacks use arrays; dynamic stacks use linked lists
- Error Management: Understanding overflow and underflow errors is crucial for proper stack implementation
- Practical Applications: Stacks are fundamental to many real-world computing scenarios beyond just undo operations

Lecture 10

Fundamentals of Queue (Q) Data Structure

This lecture introduces Queue (Q), another essential linear data structure, following previous discussions on arrays, linked lists, and stacks. The session is divided into two parts: fundamentals and types/applications of queues.

Overview and Real-World Concept

A queue operates on the principle of First Come First Served (FCFS) or First In First Out (FIFO). Using the example of a movie theater popcorn counter, elements are processed in the order they arrive—whoever arrives first is served first, regardless of what they purchase. This principle mirrors real-life queue behavior.

Queue Definition and Characteristics

- Queue is a linear data structure where elements are arranged in order of arrival
- First element stored remains at the front of the queue
- Latest elements are added at the rear
- Retrieval process starts from the front and moves toward the rear
- Alternative terminology: While stacks are Last In First Out (LIFO), queues are FIFO structures

Primary Queue Operations

Enqueue (NQ)

- Adds an element at the rear (end) of the queue
- When the queue is empty, the first element occupies both front and rear positions
- Front pointer remains stationary; rear pointer

advances with each insertion

Dequeue (DQ)

- Removes and returns the element at the front of the queue
- Front pointer moves forward to the next element
- Rear pointer remains unchanged
- Unlike the push and pop operations in stacks, dequeue requires no parameters

Peek/Front Operation

- Displays the value at the front of the queue
- Does not remove the element
- Returns the current front value without modifying the queue

Rear/Back Operation

- Displays the value at the rear of the queue
- Does **not remove** the element
- Indicates the most recently added element

Size Operation

- Returns the total number of elements currently in the queue

Queue Errors

Queue Overflow

- Occurs when attempting to enqueue an element into a full queue
- Specific to static queues with fixed capacity
- Example: Multiple users attempting to book tickets on IRCTC simultaneously, overwhelming the server's request queue

Queue Underflow

- Occurs when attempting to dequeue from an empty queue
- Happens when no elements are available for removal
- Example: Streaming data buffer becomes empty due to internet disconnection, causing video playback to freeze

Static vs. Dynamic Queues

Static Queue

- Fixed size determined at initialization
- Typically implemented using arrays
- Limited capacity, susceptible to overflow errors
- Used in hardware-constrained environments

Dynamic Queue

- **Variable size** that expands as needed
- Typically implemented using **linked lists**
- Can grow to accommodate more elements
- Better suited for applications with unpredictable data volumes

Practical Applications

Static Queue Applications

- **Wi-Fi Routers**: Data packets are stored in fixed-size queues; network congestion causes queue overflow, leading to dropped packets using the "tail drop mechanism"
- **Keyboard and Mouse Buffers**: Hardware buffers with fixed capacity; rapid inputs exceeding processing speed result in dropped keystrokes or mouse clicks

- Vending Machines: Fixed hardware capacity; each drink type occupies a specific queue with predetermined storage limits

Dynamic Queue Applications

- Messaging Applications (WhatsApp): Local dynamic queues store typed messages when internet connectivity is lost; messages are transmitted sequentially once connection is restored, maintaining the order in which they were typed
- Streaming Services (Netflix, Spotify, Amazon Prime): Segments are buffered in dynamic memory queues, allowing playback to continue even during temporary connection interruptions
- Food Delivery Apps: Orders from thousands of customers require variable-capacity queues; dynamic implementation prevents

order loss during peak demand periods

Key Differences: Queue vs. Stack

Aspect	Queue	Stack
Order	FIFO (First In First Out)	LIFO (Last In First Out)
Insertion	Rear	Top
Removal	Front	Top
Real-life Example	Theater queue	Stack of plates

Important Concepts to Remember

- Queue elements maintain their order of arrival throughout processing
- Front pointer location changes during

dequeue operations, not during enqueue
- Peek operation is crucial for examining queue state without modifying it
- Queue implementation choice (static vs. dynamic) depends on whether data volume is predictable and whether overflow is acceptable
- Real-world systems often fail gracefully when queues become full (dropped packets, ignored keystrokes) rather than crashing

Lecture 11

Lecture on Queue Types and Applications (Part 2)

Overview

This lecture continues the discussion on queues by exploring four distinct types of queues: linear,

circular, priority, and double-ended queues (deques). Each type addresses specific use cases and operational requirements, with applications ranging from real-world scenarios to computer system processes.

Linear Queue

- Basic Operation: Elements are inserted (enqueued) at the rear and removed (dequeued) from the front
- Pointer Movement: The front pointer points to the first location where an element is stored, not necessarily the first memory location in an array
- Primary Limitation: Memory wastage occurs when the rear pointer reaches the end of the array, even if space is available at the front due to deleted elements
- Static Implementation Issue: In array-based

implementations, once the rear pointer reaches the array's end, new elements cannot be inserted, regardless of available space created by dequeue operations

Circular Queue

- Key Feature: The last position connects back to the first position, creating a circular structure
- Memory Efficiency: Reuses empty space created by dequeue operations, preventing wastage found in linear queues
- Pointer Behavior: The rear pointer can continue moving into previously occupied spaces when elements are dequeued from the front
- Implementation: Can theoretically be used indefinitely for insertion and deletion operations

- Real-World Applications:
  - Traffic light systems (red → green → yellow → red)
  - Multiplayer games with sequential turns (players rotate back to the first player after the last)
  - Music playlists with looping enabled
  - CPU task scheduling using round-robin scheduling (multiple applications receive processing time in sequence)

Priority Queue

- Operational Principle: Elements are served based on priority level rather than insertion order
- Priority Determination: Prioritization follows specific policies and is independent of when elements entered the queue
- FIFO Exception: When elements have the

same priority, standard FIFO order applies

- Real-World Applications:
  - Hospital emergency rooms (triage system prioritizes accident cases over recurring headaches)
  - Airport traffic control (long-haul flights with limited fuel receive priority over short-haul flights)
  - Railway systems (express trains like Rajdhani and Shatabdi receive green signals priority)
  - CPU task scheduling (system tasks receive higher priority than application tasks)
  - TATKAL ticket bookings (separate quota with priority access)
  - Disaster management (rescue operations prioritize individuals in immediate danger)
  - Disaster management (rescue operations prioritize individuals in immediate danger)

Double-Ended Queue (Deque)

- Flexibility: Elements can be added and removed from both the front and rear ends
- Dual Functionality: Combines characteristics of both queues (FIFO) and stacks (LIFO)
- Four Core Operations:
  - Insert front: Add element at the beginning
  - Insert rear: Add element at the end
  - Delete front: Remove element from the beginning
  - Delete rear: Remove element from the end
- Real-World Applications:
  - Browser navigation (back and forward buttons allow movement through history in both directions)
  - Word processors (undo and redo functionality)
  - Train coach management (coaches can

    - Doubly linked lists (often implement deque structures)

Queue of Queues

- Concept: A main queue containing elements that are themselves queues (sub-queues)
- Streaming Application: OTT platforms like Netflix use this structure where:
    - Main queue holds upcoming scene segments of a video
    - Sub-queues contain individual video frames or packets within each segment
    - Videos are displayed as continuous motion through rapid sequential display of static images
- Interruption Handling: When internet connection drops, the system preserves progress in the main queue but must restart from the beginning of the interrupted sub-

queue, causing videos to restart slightly before the interruption point

Key Takeaways

- Linear Queue: Regular line following FIFO (First In First Out) principle
- Circular Queue: Ring-shaped line that reuses empty spaces and prevents memory wastage
- Priority Queue: VIP line where urgency determines service order
- Deque: Open-ended line allowing entry and exit from both sides

Lecture 12

Introduction to Nonlinear Data Structures: Trees

This lecture introduces trees, the first nonlinear data structure, contrasting them with linear data

structures like arrays, linked lists, stacks, and queues where paths are predetermined. Trees feature multiple possible paths through their elements, creating hierarchical relationships between nodes.

Core Concepts and Terminology

Fundamental Definition

- A tree is a data structure where elements are arranged hierarchically rather than sequentially
- Each unit or element in a tree is called a **node**
- Nodes are connected by **edges** (lines linking parent to child nodes)

Key Node Types

- **Root node**: The first and topmost element of the tree (drawn at the top diagrammatically)
- **Child node**: A node connected below another

node
- **Parent node**: A node that has one or more child nodes connected to it
- **Leaf node**: Terminal nodes with no child nodes (found at the end of any pathway)

Essential Tree Properties

- A node can have multiple children, but each node can have only one parent
- A tree cannot have a node with two parents (distinguishes trees from graphs)
- The structure prevents circular connections

Family Tree Use Case

A practical example demonstrates tree structure using a family hierarchy:

- Grandfather (root) has three children: Son A, Son B, and Daughter

- Son A has one daughter (D1)
- Daughter has two sons (S1 and S2)
- Son B has no children

This illustrates how trees effectively represent hierarchical relationships with clear parent-child dependencies.

Tree Properties: Depth and Height

Depth (represented as lowercase d)

- Measures how far a node is from the root
- Calculated by counting the number of edges needed to reach from the node to the root
- Root node has depth = 0
- Each subsequent level increases depth by 1
- Every node has its own unique depth value

Height (represented as lowercase h)

- Represents the maximum depth value in an

entire tree
- Measures the distance from root to its deepest leaf node
- A single value that characterizes the entire tree
- Calculated by finding the node requiring the most edges to reach from root

Sub-trees

- Any section of a tree that is a child of another node constitutes a sub-tree
- A tree can be understood as multiple nested sub-trees
- Similar to how a branch of a real tree resembles a complete tree when viewed independently

Real-World Applications

Folder Structure: Computer file systems organize folders hierarchically where each subfolder belongs to only one parent folder
Corporate Hierarchy: Organizations structure employees with CEO at root, managers as intermediate nodes, and workers as leaf nodes
Book Classification: Libraries organize books through hierarchical systems—language → genre → subgenre → author → title
Biological Classification: Taxonomy arranges organisms hierarchically (animal → vertebrate → mammal → dog)
Family Trees: Represent genealogical relationships across generations (typically following patrilineal lines to maintain single-parent constraint)

Tree Traversal Methods

Tree traversal refers to the process of reading or

visiting all nodes in a tree. The non-linearity of trees creates multiple valid traversal paths, unlike linear structures.

Depth-First Traversal Methods

Depth-first traversals follow the logic until reaching the deepest leaf nodes before backtracking. The position of the root node determines the traversal type.

Pre-order Traversal (Root, Left, Right)

- Root is visited first
- Then traverses left subtree
- Finally traverses right subtree
- Example: A, B, D, E, C

In-order Traversal (Left, Root, Right)

- Left subtree is visited first
- Then root node
- Finally right subtree

- Typically works clearly only when nodes have at most two children
- Example: D, B, E, A, C

Post-order Traversal (Left, Right, Root)

- Left subtree is visited first
- Then right subtree
- Root node is visited last
- Example: D, E, B, C, A

Key Rule: In all traversals, left subtree always comes before right subtree

Breadth-First Traversal (Level-Order)

- Starts from root at top level
- Proceeds to next level, then next, working downward
- At each level, traversal moves left to right
- Unlike depth-first methods, this stays at each level before descending

- Example: A, B, C, D, E

Traversal Characteristics

- All traversals follow the principle that left branches are processed before right branches
- Depth-first methods require careful attention to root position
- Level-order traversal is visually intuitive but algorithmically requires different implementation
- Traversal methods are essential for reading and processing tree data due to multiple valid paths

Key Takeaways

- Trees organize data hierarchically with single-parent constraint per node

- Depth applies to individual nodes; height applies to entire tree
- Multiple traversal methods exist due to non-linear structure
- Each traversal method serves different algorithmic purposes
- Trees are fundamental for representing hierarchical relationships in real-world applications
- Future sessions will cover specialized tree types used in algorithms

Lecture 13

Linear Data Structures: Stacks and Queues

Overview of the Lecture

This lecture continues the study of linear data structures, covering two fundamental types: stacks and queues. While arrays allow indexed

access and linked lists use connections between elements, stacks and queues have unique operational characteristics that make them essential for specific use cases.

Stacks (LIFO - Last In First Out)

Definition and Concept

A stack is a linear data structure where the most recently added element is the first one to be removed, following the LIFO (Last In First Out) principle. This differs from queues which follow FIFO principles.

Real-World Examples

- Wedding plates: Fresh plates are placed on

top; the most recently placed plate is taken first

- Document editing (Undo function): The last change made is the first one undone when using Ctrl+Z
- Newspaper stacks: The latest newspaper sits on top
- Browser history: Recent pages visited are accessed first

Basic Stack Operations

- Push: Adds a new element to the top of the stack
- Pop: Removes and returns the top element from the stack
- Peek: Views the top element without removing it
- Size: Returns the number of elements currently in the stack

Stack Implementation Approaches

Static Stack

- Fixed size determined at creation
- Implemented using arrays
- Memory-efficient for predictable sizes
- Examples: Adobe Photoshop (50 undo operations), Microsoft Excel (100 undo operations)

Dynamic Stack

- Size can grow as needed
- Implemented using linked lists
- Provides flexibility but requires more memory management
- Example: Microsoft Word (theoretically unlimited undo, limited by available RAM)

Stack Errors

Stack Overflow

- Occurs when attempting to push an element onto a full static stack
- Example: Opening too many browser tabs causes system memory stack to overflow
- System cannot allocate additional space for new elements

Stack Underflow

- Occurs when attempting to pop from an empty stack
- Example: Clicking the back button in a browser with no history
- No element exists to return

Applications of Stacks

- Undo/Redo functionality in applications
- Function call management in programming

- Expression evaluation and parsing
- Depth-first search algorithms
- Browser navigation history

Queues (FIFO - First In First Out)

Definition and Concept

A queue is a linear data structure where the first element added is the first one to be removed, following the FIFO (First In First Out) principle. Elements are inserted at the rear end and removed from the front end.

Real-World Examples

- Movie ticket queues: People are served in the order they arrive

- Network printer queues: Print jobs are processed in the order they are submitted
- Restaurant service: Customers are served based on arrival order (when properly managed)

Basic Queue Operations

- Enqueue (NQ): Adds an element to the rear end of the queue
- Dequeue (DQ): Removes and returns the front element
- Peek (Front): Views the front element without removing it
- Peek (Rear): Views the rear element
- Size: Returns the number of elements in the queue

Queue Types

Linear Queue

- Simple implementation with front and rear pointers
- Memory wastage problem: When elements are dequeued, the front moves forward but empty cells cannot be reused
- Implemented using arrays

Circular Queue

- Logically arranged in a circular manner (not in physical memory)
- Solves the memory wastage problem of linear queues
- When the rear pointer reaches the end, it wraps around to the beginning if space is available
- Implemented using linked lists
- Applications: Traffic lights, multiplayer games (turn-based), music playlists

Double-Ended Queue (Deque)

- Allows insertion and deletion from both ends
- Combines properties of stacks and queues
- Can add/remove from front or rear
- Applications: Browser forward/back buttons, undo/redo functionality

Priority Queue

- Elements are served based on priority rather than strict FIFO order
- Higher priority elements are served first
- If priorities are equal, FIFO order is maintained
- Applications: CPU task scheduling, hospital emergency rooms (triage), airline boarding

Queue Errors

Queue Overflow

- Occurs when a static queue is full and a new

element is attempted to be added
- Example: Concert ticketing system becomes full during high-demand periods
- Server may reject new requests, causing apparent system hang

Queue Underflow

- Occurs when attempting to dequeue from an empty queue
- Example: Video streaming freezes when buffer queue becomes empty
- Happens when internet connection is lost during streaming

Queue Implementation Approaches

Static Queue

- Fixed size, implemented using arrays
- Applications: Network printer buffers, keyboard buffers (16-64 bytes), Wi-Fi router

packet queues

Dynamic Queue

- Variable size, implemented using linked lists
- Applications: WhatsApp message queues, streaming services (Netflix, YouTube, Hotstar)

Real-World Applications

Network Printing

- Print jobs queue at the printer in FIFO order
- Printer has fixed memory to store pending jobs

Streaming Services

- Video is essentially a series of consecutive images played in sequence
- Uses queue of frames at receiver end (client buffering)

- Uses multiple queues (queue of queues) for different video segments
- When connection is slow, buffering pauses and older frames are played

CPU Task Scheduling

- Round Robin scheduling: CPU allocates time slice to each task, then moves to next task
- Shortest Job First: Tasks with shorter execution time get priority
- Continuous task switching creates perception of parallel processing

Keyboard and Mouse Input

- Input buffer stores keystrokes in a queue
- If typing speed exceeds processing speed, buffer can become full
- Static buffer typically 16-64 bytes

Traffic Management

- Traffic lights follow circular queue pattern: Red → Green → Yellow → Red
- Vehicle queues at intersections follow FIFO order
- GPS navigation uses queue-based routing systems

Online Multiplayer Games

- Turn-based games (Ludo): Player turns cycle in circular queue order
- Each player's turn is served in sequence

Disaster Management

- Triage system uses priority queues in emergency situations
- Life-saving operations get highest priority
- Resource allocation follows priority-based queuing

Comparison: Stack vs Queue

Aspect	Stack	Queue
Order	LIFO (Last In First Out)	FIFO (First In First Out)
Insertion	Top (push)	Rear (enqueue)
Deletion	Top (pop)	Front (dequeue)
Primary Use	Undo/Redo, function calls	Scheduling, buffering
Typical Implementation	Array or Linked List	Array or Linked List

Important Notes on Implementation

- **Logical vs Physical Memory**: While stacks and queues appear as ordered structures in diagrams, they are physically stored in flat memory locations. The operating system and applications manage the logical structure through pointers and indices.
- **Not Basic Data Types**: Stacks, queues, and other advanced data structures are not primitive data types in most programming languages. They are implemented using arrays or linked lists with specific operational constraints.
- **Memory Limitations**: Both static stacks and queues are limited by available RAM.

Dynamic implementations grow as needed but are ultimately constrained by system memory.

Assessment Information

- Quiz Date: Weekend of October 3rd
- Question Types: Multiple Choice Questions (MCQ), True/False, matching, or short answer fill-in-the-blanks
- Weightage: Quizzes count for 40% of total marks
- Quiz Count: Three quizzes total, with best two scores counted
- Syllabus Coverage: Quiz covers Module 1 and Module 2 (Linear Data Structures) only

Lecture 14

Introduction to Non-Linear Data Structures: Trees and Their Applications

Overview

This lecture continues the exploration of non-linear data structures, specifically focusing on different types of trees and their practical applications in computer science. The session builds upon fundamental tree concepts including hierarchical arrangement of elements, tree properties (depth and height), and traversal methods covered in the previous session.

Recap of Tree Fundamentals

- Tree Definition: A data structure where elements are arranged hierarchically, with each node capable of having multiple

children, but each child having only one parent

- Key Properties:
  - Depth: The distance of a node from the root (number of jumps required)
  - Height: The distance from a node to its deepest leaf
- Traversal Methods: Depth-first traversals (3 types) and breadth-first traversal

General Trees

Definition: A tree where nodes can have any number of children with no fixed minimum or maximum constraints.
Characteristics:

- No restriction on the number of children per node
- Variable branching factor across different nodes

Real-World Applications:

- Computer folder and directory structures
- Family trees (patrilineal representation)
- Organizational hierarchies with variable reporting structures
- Biological taxonomy and classification systems

Binary Trees

Definition: A tree where each node can have at most two children (meaning zero, one, or two children are possible).
Key Concept: "At most" means the maximum is fixed at two, but the minimum is not restricted.
Characteristics:

- Maximum of two children per node
- Nodes can have zero, one, or two children
- Still classified as binary tree if some nodes

have fewer than two children

Real-World Applications:

- Tournament knockout structures (sports competitions where two teams compete, winners advance)
- Decision trees and flowcharts with yes/no decision points
- Binary decision processes in systems (ATM transactions, quality checks)

Complete Binary Trees

Definition: A binary tree where all levels except the last are completely filled with exactly two children per node, and the last level is filled from left to right.

Key Rules:

- All levels before the last must have exactly

two children per node
- Last level can have fewer than two children
- Last level must be filled left to right (no gaps)
- If a node at the last level has one child, it must be the left child

Structural Requirement: Ensures efficient storage and minimal wasted space in tree representation.

Binary Search Trees (BST)

Definition: A binary tree with an additional ordering property where values matter and must follow a specific arrangement pattern.
Fundamental Property:

- Left subtree values are always less than the parent node value
- Right subtree values are always greater than the parent node value

- This property holds recursively for all subtrees

Value Location Patterns:

- Minimum value: Found by traversing left continuously from root until reaching a leaf
- Maximum value: Found by traversing right continuously from root until reaching a leaf

Applications:

- Dictionary searches in traditional and digital formats
- Database indexing and efficient data retrieval
- Student ID lookups where IDs follow systematic templates
- Any scenario requiring frequent search operations

Limitations:

- If data is inserted in sorted order (increasing or decreasing), the tree becomes highly unbalanced
- Unbalanced trees behave like linked lists, requiring sequential traversal
- This defeats the efficiency advantage of binary search tree structure

AVL Trees (Self-Balancing Binary Search Trees)

Definition: A self-balancing binary search tree where the difference in heights between left and right subtrees cannot exceed one for any node.
Balancing Rule: For any node, |height(left subtree) - height(right subtree)| ≤ 1
Key Operation - Rotation:

- Automatic rebalancing operation performed during insertion

- Conceptually similar to restructuring a hanging string and ball system
- Redistributes nodes to maintain balance without violating BST properties
- Ensures tree remains balanced during data entry

How Rotation Works:

- When an imbalance is detected (difference > 1), the middle node is promoted to a higher level
- Child nodes are rearranged accordingly
- The tree structure is modified while maintaining BST ordering property

Advantages:

- Reduced number of hops during search operations compared to unbalanced trees
- Guaranteed efficient search performance
- Useful for in-memory indexing in databases

- Applied in library catalog systems where new disciplines must be accommodated

Real-World Application: Library cataloging systems use similar rotation concepts where new subjects and subdisciplines can be inserted within existing hierarchies without requiring complete reorganization.

Heaps

Definition: A complete binary tree with additional ordering constraints based on parent-child value relationships.
Important Distinction: A heap is NOT a binary search tree; it does not require left subtree values to be less than parent or right subtree values to be greater.
Two Types of Heaps:

1. Max Heap:

    - Parent value is always greater than or equal to its children
    - Root node contains the maximum value in the entire tree
    - Property holds recursively for all subtrees
2. Min Heap:
    - Parent value is always less than or equal to its children
    - Root node contains the minimum value in the entire tree
    - Property holds recursively for all subtrees

Implementation Details:

- During insertion, values are rearranged to maintain the heap property
- Structure remains a complete binary tree after each insertion
- Values are reordered (not repositioned structurally) to satisfy parent-child constraints

Primary Application - Priority Queues:

- Max heap: Higher priority elements are processed first (higher numbers get higher priority)
- Min heap: Lower priority number elements are processed first (priority 1 before priority 2)
- Medical example: Emergency patients (high priority) are attended before routine cases
- System processes: System-level tasks take precedence over user-level applications

Comparison Summary

Tree Type	Children Limit	Value Ordering	Key Property
General Tree	No limit	Not required	Hierarchical

			structure only
Binary Tree	Max 2	Not required	At most 2 children per node
Complete Binary Tree	Max 2	Not required	All levels full except last; last level filled left-to-right
Binary Search Tree	Max 2	Required	Left < parent < right for all nodes
AVL Tree	Max 2	Required	BST +

			height differenc e ≤ 1
Heap	Max 2	Partial ordering	Complete binary tree + parent-child value relations hip

Key Takeaways

- Tree structure choice depends on specific application requirements
- Balance is critical for search efficiency in binary trees
- Value ordering becomes important when

search operations are frequent
- Heap structures enable efficient priority queue implementations
- Self-balancing trees (AVL) automatically maintain efficiency by preventing degenerate cases
- Different tree types have evolved to address specific computer science needs in data retrieval, prioritization, and hierarchical representation

Lecture 15

Understanding Graphs: A Non-Linear Data Structure

Overview

This lecture introduces graphs as a non-linear data structure used to capture relationships

between nodes, contrasting them with trees which represent hierarchical structures. Graphs are more suitable when modeling realistic scenarios where nodes can have multiple connections and relationships.

Trees vs Graphs: Key Differences

Trees:

- Represent hierarchical relationships
- Each node (except root) has exactly one parent
- Single-sided perspective (useful for organizational charts or file systems)
- Limited to parent-child relationships

Graphs:

- Represent relationships between equal entities
- Nodes can have multiple connections

- Capture complex, multi-directional relationships
- More suitable for realistic scenarios with interconnected entities

Example: A complete family tree considering both parental sides requires a graph structure because children have two parents, violating the tree constraint of single parent per child.

When to Use Trees vs Graphs

Scenario	Data Structure	Reason
Company hierarchy (people and designations)	Tree	Clear reporting structure with single supervisor

Corporate group with independent companies	Graph	Companies are autonomous with multiple interconnections
Institute collaborations and networks	Graph	Institutions are independent with multiple research partnerships
Department and research groups within institution	Tree	Clear hierarchical structure from institution to department to groups

Graph Components and Representation

Basic Elements:

- Vertices (V): Objects or entities in the graph (singular: vertex, plural: vertices)
- Edges (E): Connections between vertices representing relationships

Standard Notation: G = (V, E)

- V = set of all vertices
- E = set of all edges connecting vertices

Example:

```
V = {A, B, C, P, Q, R, S, X, Y}
E = {(C,S), (A,P), (A,Q), (B,R), (S,P),
(Q,R), (S,X), (P,X), (Q,Y), (R,Y), (X,Y)}
```

Graph Properties

Degree of a Vertex:

- The number of edges directly connected to a vertex
- Calculated by counting all edges emanating from that vertex

Example: In a family graph, vertex P connected to A, S, and X has a degree of 3

Types of Relationships: Graphs can represent various relationship types beyond hierarchy:

- Spouse relationships
- Cousin relationships
- Collaboration partnerships
- Any multi-directional connection

Key Takeaways

- Definition: Graphs capture relationships between nodes without imposing hierarchical constraints
- Representation: Graphs are formally defined as G = (V, E) where V lists vertices and E lists edges
- Flexibility: Nodes can connect to any number of other nodes
- Practical Application: Graphs model real-world networks like social media, family structures, organizational collaborations, and transportation systems more accurately than trees

Lecture 16

Graph Properties and Relationship Representation in Networks

Overview

This lecture explores how relationships and connections are depicted in graphs, focusing on the key properties that distinguish different types of graphs. The discussion emphasizes practical applications in social media analysis and network structures.

Directed vs. Undirected Graphs

Directed Graphs:

- Relationships are defined in a specific direction only
- The opposite relationship is not automatically true
- Example: "A is a son of B" does not mean "B is a son of A"
- Different relationships in opposite directions are considered separate (e.g., "son of" vs.

"father of")
- Use directed graphs when direction of relationship is important

Undirected Graphs:

- Relationships have no specific direction
- If a relationship exists in one direction, it is true in both directions
- Example: "A is a cousin of B" means "B is a cousin of A" (mutual relationship)
- Use undirected graphs when interested only in connections without directional importance

Social Media Examples:

- Facebook friendship: Undirected graph (mutual friendship once request is accepted)
- Instagram follows: Directed graph (following is not reciprocal)
- LinkedIn connections: Both types

(connections are undirected, follows are directed)

In-Degree and Out-Degree

In Degree:

- Number of edges coming into a vertex
- In social media: number of users or pages that follow you

Out Degree:

- Number of edges going out from a vertex
- In social media: number of users or pages that you follow

Practical Application - Instagram Influencer Marketing:

- In-degree is crucial for influencer selection
- Influencers with high in-degree (many

followers) guarantee audience for marketing
- Out-degree (who they follow) is not relevant for marketing purposes
- Marketing firms prioritize users with high in-degree

Weighted vs. Unweighted Graphs

Unweighted Graphs:

- All connections have equal value
- Example: "A knows B" treated the same as any other knowing relationship
- No strength or value assigned to connections

Weighted Graphs:

- Each connection has a strength value based on context
- Strength varies depending on relationship type and interaction frequency

- Examples:
  - Family relationships: Siblings have stronger connection than distant cousins
  - Flight networks: More flights between destinations = higher weight
  - Social media recommendations: Frequently watched content has higher weight

YouTube Recommendation Example:

- High weight: Videos from subscribed channels watched frequently result in stronger connections and more recommendations
- Low weight: New, unwatched channels or one-time recommendations have minimal weight
- Recommender systems prioritize content with high connection strength

Connected vs. Disconnected Graphs

Connected Graphs:

- Every vertex can be reached from every other vertex
- Paths may vary in length
- Example: Within a community, members are connected through direct or indirect relationships

Disconnected Graphs:

- Certain vertices or subgraphs may be isolated
- No connection exists between some sections
- Example: Port Blair (island) has no road connection to mainland India

Practical Example:

- Amdabad to Silchar (road network): Connected graph
- Amdabad to Port Blair (road network only): Disconnected graph

Cyclic vs. Acyclic Graphs

Cyclic Graphs:

- Possibility exists to return to the starting vertex through different paths
- Multiple pathways connect vertices, creating loops
- Example: A-B-C-A (can return to starting point without retracing steps)
- Social example: Friend networks where A is friends with B and C, and B is friends with C

Acyclic Graphs:

- No path returns to the starting vertex

without retracing the same route
- Terminal endpoints exist in the network
- Example: Parent-child relationships (cannot return to parent if moving forward only)
- Social example: Organizational hierarchies or metro networks with terminal stations

Practical Examples:

- Airport network with return flights: Cyclic graph (flights exist in both directions between cities)
- Metro line network: Acyclic graph (terminal stations prevent returning without retracing paths)

Summary of Graph Properties

Graphs can be characterized by multiple independent properties:

- Direction: Directed or undirected
- Weight: Weighted or unweighted
- Connectivity: Connected or disconnected
- Cycles: Cyclic or acyclic

These properties work together to represent complex real-world networks and relationships in social media, transportation systems, and other domains.

Lecture 17

Graph Traversals: Depth First Search and Breadth First Search

Overview

This lecture covers graph traversal methods, specifically Depth First Search (DFS) and

Breadth First Search (BFS). Unlike trees where hierarchy is emphasized and a child can have only one parent, graphs represent relationships where nodes can have multiple connections. Graph traversals are used to search for elements within a graph structure.

Key Differences Between Graphs and Trees

- Graphs capture relationships and connections between nodes; a node can have multiple parents
- Trees represent hierarchy with single parent-child relationships
- Graphs do not have a designated root node; traversal can start from any node
- The terms "search" and "traversal" are often used interchangeably since traversal is performed to search for specific elements

Depth First Search (DFS)

Definition and Concept:

- Explores a graph by going as far as possible along one path before backtracking
- Visits the first unvisited neighbor and follows it further down until reaching a dead end
- Once no more unexplored connections exist, the algorithm backtracks one level and repeats

Applications:

- Finding if any connection exists between two nodes
- Useful in social network analysis to determine if a person is connected within a network
- Can establish connections through multiple

degrees (friend, friend of friend, etc.)

Algorithm Implementation Using Stack:

1. Push the starting node onto the stack
2. Pop the top node and mark it as visited
3. Find all connected unvisited neighbor nodes of the popped node and push them onto the stack
4. Repeat steps 2-3 until the stack is empty

Characteristics:

- Multiple valid DFS paths exist depending on which neighbor is visited first
- No predetermined order (left-first, right-first) as in tree traversals
- Both possible paths are correct solutions

Breadth First Search (BFS)

Definition and Concept:

- Explores a graph by visiting all nodes at the current depth level before moving to the next depth level
- Starts with a given starting node and progressively expands outward
- Visits nodes level by level: level 0, then all level 1 nodes, then level 2 nodes, etc.

Applications:

- Determining the degree of separation between nodes
- Finding the shortest path between two nodes
- Identifying how many connections (hops) separate two people in a network
- Based on the concept of "six degrees of separation"

Algorithm Implementation Using Queue:

1. Enqueue the starting node into the queue

2. Dequeue the front node and mark it as visited
3. Find all connected unvisited neighbor nodes of the dequeued node and enqueue them
4. Repeat steps 2-3 until the queue is empty

Characteristics:

- Like DFS, multiple valid BFS paths exist based on the order neighbors are visited
- Maintains consistent level-by-level traversal regardless of visual graph layout
- All valid paths following the algorithm correctly are acceptable solutions

Comparison of DFS and BFS

Aspect	DFS	BFS
Data Structure	Stack	Queue

Strategy	Go deep along one path, then backtrack	Visit all nodes at current depth first
Use Case	Finding if any connection exists	Finding degree of separation
Order	Path-dependent	Level-dependent

Important Implementation Notes

- Understanding stack and queue operations (push, pop, enqueue, dequeue) is essential for implementing these traversals
- Unlike tree traversals which have pre-order, post-order, and in-order variants, graph traversals have only two major types: DFS

and BFS
- Starting node is specified in advance; unlike trees, graphs have no inherent root node
- Both DFS and BFS can be started from any node in the graph, leading to multiple valid traversal paths

Lecture 18

Introduction to Non-Linear Data Structures

This lecture covers the fundamental concepts of non-linear data structures, specifically trees and graphs, which are essential for representing hierarchical and interconnected data in real-world applications.

Overview of Linear vs. Non-Linear Data Structures

Linear Data Structures:

- Arrays, linked lists, stacks, and queues allow sequential access to elements
- Elements are accessed one by one in a predetermined order
- Examples: Array indexing (0, 1, 2..) or following links in a linked list
- Similar to following a single path or receiving guidance from one person

Non-Linear Data Structures:

- Trees and graphs allow multiple pathways and connections
- Multiple options exist at each point in the structure
- Better representation of real-world scenarios with choices and alternatives
- Example: Educational path selection (online/offline → college selection → degree choice)

Trees: Hierarchical Structure

Basic Concept and Definition

- A tree is a hierarchical data structure with a root node at the top and branches extending downward
- Follows a reverse structure compared to real trees (root at top, leaves at bottom in computer science)
- Fundamental property: **A child node can have only one parent node**
- One parent can have multiple child nodes

Key Terminology

Tree Components:

- **Root Node**: The top-most node with no parent
- **Parent Node**: A node that has child nodes

connected to it
- Child Node: A node that is connected to a parent node
- Leaf Node: A terminal node with no children
- Edges: The connections between parent and child nodes
- Subtree: Any subset of nodes and edges forming a smaller tree within the larger tree

Tree Measurements:

- Depth: Number of edges from the root to a specific node (root has depth 0)
- Height: The maximum depth of any leaf node in the tree (also equal to the number of edges from root to the deepest leaf)

Real-World Applications of Trees

- Family Trees: Hierarchical representation of ancestral relationships (patriarchal structure)

- Corporate Hierarchy: CEO → Managers → Team Leaders → Workers showing organizational structure
- Folder Structure: Computer file organization (Program folder → Semester folder → Course folder)
- Library Classification System: Genre → Sub-genre → Author → Books (hierarchical organization system)
- Biological Classification: Kingdom → Phylum → Class → Order → Family → Genus → Species (taxonomy)

Tree Traversal Methods

Depth-First Traversal (DFS): Continues down one path until reaching a leaf, then backtracks to explore other paths

Three variations based on root position:

- Pre-order Traversal: Root → Left → Right

    - Example: A B D C (visit root first, then left subtree, then right subtree)
- In-order Traversal: Left → Root → Right
    - Example: D B E A C (visit left subtree, root, then right subtree)
    - Implementation: Uses Stack data structure
- Post-order Traversal: Left → Right → Root
    - Example: D E B C A (visit left subtree, right subtree, then root)

Breadth-First Traversal (Level-order): Visits nodes level by level from top to bottom, left to right

- Example: A B C D E (level 1, then level 2, then level 3)
- Implementation: Uses Queue data structure

Graphs: General Connectivity Structure

Definition and Components

A graph is a more general non-linear data structure that allows multiple parent-child relationships and circular connections.

Graph Representation:

- Mathematically represented as G = (V, E)
- V (Vertices): Set of nodes in the graph
- E (Edges): Set of connections between vertices

Terminology:

- Vertex: A node in the graph (equivalent to node in tree)
- Edge: A connection between two vertices
- Degree: Number of edges connected to a vertex

Key Differences Between Trees and Graphs

Property	Tree	Graph
Parent-Child Relationship	One parent per child	Multiple connections allowed
Root Node	Has a root	No fixed root
Cycles	No cycles possible	Cycles can exist
Terminology	Parent, child, ancestor	Vertices, edges, neighbors

Types of Graphs

Directed vs. Undirected:

- Undirected Graph: Edges have no direction; relationship is bidirectional (e.g., two-way roads, mutual friendship)

- Directed Graph: Edges have direction; relationship is one-way (e.g., one-way traffic, following on social media)
  - In-degree: Number of edges coming into a vertex
  - Out-degree: Number of edges going out from a vertex
  - Social media influencer example: High in-degree (many followers) vs. out-degree (following others)

Weighted vs. Unweighted:

- Unweighted Graph: All edges have equal value
- Weighted Graph: Edges have different values/weights representing strength, distance, or cost (e.g., road distances, relationship closeness)

Connected vs. Disconnected:

- Connected Graph: All vertices are reachable from any other vertex through some path
- Disconnected Graph: Some vertices cannot be reached from others (isolated components)
- Example: Road connectivity in India (mainland connected, but islands disconnected)

Cyclic vs. Acyclic:

- Cyclic Graph: Contains at least one path that starts and ends at the same vertex
- Acyclic Graph: Contains no cycles; is a tree structure

Real-World Applications of Graphs

- Social Networks: Connections between people, influencer networks for marketing
- Road Networks: Cities as vertices, roads as

edges (weighted with distances)

- **Web Pages**: Pages as vertices, hyperlinks as directed edges
- **Biological Networks**: Species relationships, food chains
- **Recommendation Systems**: Users and products connected based on preferences

Graph Traversal Methods

Depth-First Search (DFS): Explores as far as possible along one branch before backtracking

- Process:
  A. Start at a vertex and push it onto a stack
  B. Pop the top vertex from stack and mark as visited
  C. Find all unvisited neighbors of the popped vertex
  D. Push unvisited neighbors onto the stack
  E. Repeat until stack is empty

- Implementation: Uses Stack data structure
- Use case: Checking if certain nodes are reachable; finding connected components

Breadth-First Search (BFS): Explores all immediate neighbors before moving to the next level

- Process:
  A. Start at a vertex and enqueue it into a queue
  B. Dequeue a vertex from the front and mark as visited
  C. Find all unvisited neighbors of dequeued vertex
  D. Enqueue unvisited neighbors into the queue
  E. Repeat until queue is empty
- Implementation: Uses Queue data structure
- Use case: Finding shortest paths; exploring

Practical Example: DFS with Stack Implementation

For a graph starting from vertex A:

1. Push A, pop and mark visited
2. Find neighbors P and Q, push both
3. Pop P, mark visited, find neighbor X, push it
4. Pop X, mark visited, find neighbor S, push it
5. Continue until all vertices are visited and stack is empty

- Result: Visits vertices in depth order (A → P → X → S → C → Q → Y → R → B)

Practical Example: BFS with Queue Implementation

For the same graph starting from vertex A:

1. Enqueue A, dequeue and mark visited
2. Find neighbors P and Q, enqueue both

Dequeue P, mark visited, find neighbor X, enqueue it

4. Dequeue Q, mark visited, find neighbor Y, enqueue it
5. Continue level by level

- Result: Visits vertices by level (A → P, Q → X, Y → S, R → C, B)

Key Takeaways

- Trees are specialized graphs with one-parent constraint, useful for hierarchical data
- Graphs are more general structures allowing multiple connections and cycles
- Traversal methods (DFS and BFS) differ in approach but both can be implemented using linear data structures (Stack and Queue respectively)
- Tree traversals vary by node visit order (pre-order, in-order, post-order)

connected) determine their real-world applicability
- Linear data structures (stack and queue) support traversal of non-linear structures, demonstrating their interconnected importance

Lecture 19

Hash Data Structures and Their Applications

Overview

Hash data structures are an essential nonlinear data structure that have become increasingly important in modern computing. This lecture covers hashing fundamentals, hash table implementation, and various applications of hashing in database indexing and cybersecurity.

Functions and Basic Concepts

Understanding Functions

- Functions are organized steps that take inputs and produce outputs based on predefined operations
- Functions are called with input parameters in parentheses and return a value as output
- Output length can vary depending on the function type and input

Key Difference - Fixed Length Output

- Standard functions (like square or uppercase) produce variable-length outputs depending on the input
- Hash functions always produce fixed-length output regardless of input size
- This fixed-length output is also called a "digest" or "message digest"

- Hash function outputs are typically alphanumeric (containing both numbers and letters)

Modulo Function and Hash Calculation

Understanding Modulo (MOD) Operation

- Modulo function divides a number by X and returns the remainder as output
- Syntax: number mod X
- Examples:
  - 101 mod 10 = 1 (remainder when 101 is divided by 10)
  - 27 mod 5 = 2 (remainder when 27 is divided by 5)
  - 25 mod 5 = 0 (no remainder)

Determining Hash Values

- The value of X in modulo function

determines the number of unique hash values

- If you want 10 different hashes, use mod 10
- If you want 100 different hashes, use mod 100
- Practical application: mod 10 extracts the last digit of a number

Hash Tables and Key-Value Pairs

Hash Table Structure

- Stored as key-value pairs (technically index-key-value pair)
- Key: unique identifier used to identify records (e.g., roll number)
- Value: data associated with the key (e.g., student name)
- Index: the hash value that points to a location in the table

Example Application

- Roll number serves as the key
- Student name and details serve as the value
- Using mod 10, the index is determined by the last digit of the roll number
- Students with roll numbers ending in the same digit are grouped together

Hash Collision and Resolution

What is Hash Collision?

- Occurs when the hash function returns the same hash value for two different inputs
- Example: Roll numbers B26-SME-001 and B26-SME-011 both end in 1, creating a collision at index 1

Collision Resolution - Hash Chaining

- Uses linked lists to handle multiple records

with the same hash value
- When a collision occurs, new records are chained to existing records at that index
- Allows efficient storage and retrieval of multiple keys with identical hash values

Properties of Good Hash Functions for Indexing

Essential Characteristics

- Fast Calculation: Hash computation should be quick to enable rapid searches
- Consistency: Same input should always produce the same hash value
- Even Distribution: Keys should be spread evenly across different hash positions
- Fewer Collisions: While some collisions are inevitable, they should be minimized
- Effective Key Usage: All information in the

key should be utilized for searching purposes

Hash Functions in Cybersecurity

One-Way Property

- If input is hashed to produce output, the original input cannot be recalculated from the hash
- Critical for securing sensitive information like passwords

Avalanche Effect

- Even a single-digit change in input must produce a completely different hash
- Prevents attackers from guessing the original input based on similar hashes
- Ensures security by making hash outputs unpredictable

Collision Resistance

- For cybersecurity purposes, each unique input should ideally produce a unique hash
- No two different inputs should produce the same hash value
- Essential for maintaining data integrity and security

Computing Speed Considerations

- For integrity checks: Hash calculation should be fast
- For password storage: Hash calculation should be deliberately slow to prevent password cracking
- Speed requirements vary based on the application

Password Storage Using Hashing

How Password Storage Works

- User passwords are never stored in their original format
- Instead, the hash of the password is calculated and stored in the database
- When a user logs in, their entered password is hashed and compared with the stored hash
- If hashes match, the password is correct; if not, access is denied

Security Advantages

- Even if the database is compromised, passwords cannot be recovered from hashes
- The one-way property ensures attackers cannot reverse-engineer the original password
- Avalanche effect means typing a password incorrectly will produce a different hash

Integrity Checks Using Hash Functions

Purpose

- Verify that data or files have not been altered or corrupted during transmission

Process

1. Sender calculates hash of original data/file
2. Hash and data are packaged together and sent to receiver
3. Receiver unpacks the package and separates data from the hash
4. Receiver recalculates hash based on received data
5. Calculated hash is compared with received hash
6. If hashes match, data integrity is confirmed; if not, data is deemed corrupted

Advantage

- Fast hash calculation is suitable for this use case
- Provides quick verification of data integrity without opening potentially corrupted files

Key Takeaways

- Hash functions always produce fixed-length output, distinguishing them from standard functions
- Hash indexing enables faster database searches through organized placement of records
- Hash collision can be efficiently resolved using chaining with linked lists
- Properties required for hash functions vary based on their application
- Simpler hash functions with potential

collisions are suitable for indexing
- Complex hash functions with collision resistance are essential for cybersecurity applications
- Password security relies on one-way hashing to prevent password recovery
- Integrity checks use hash functions to detect data tampering or corruption

Lecture 20

Conclusion of Non-Linear Data Structures: Introduction to Blockchain

Overview

This lecture concludes the non-linear data structures module by exploring blockchain technology, which applies concepts from linked lists, hashing, trees, and graphs. Blockchain is

introduced as a practical application that combines multiple data structure concepts to create a secure, decentralized system for storing and managing transactions.

Key Concepts Review

Linked Lists

- Linear data structure where each node points to the next node's memory address
- Nodes are added sequentially, typically starting from a head node
- Example: Music playlists function as linked lists
- Reading proceeds forward from the head node

Cryptographic Hash Functions

- Takes variable-length input and produces

fixed-length output
- Key properties include:
  - Collision-free: Unique inputs produce unique outputs
  - Non-reversible: Cannot calculate input from output
  - Sensitive: Even changing a single bit completely changes the output

Blockchain Structure and Components

Basic Block Structure

- Header: Contains hash of previous block, hash of current block, and Merkle root
- Transactions: Set of operations stored chronologically within each block
- Genesis Block: First block in a blockchain (equivalent to head node in linked lists)
- Chain Formation: Each block stores the hash

of the previous block, creating logical connections

How Blocks Connect

- Unlike linked lists that use memory addresses, blockchain uses cryptographic hashes
- Each block contains: previous block's hash, current block's hash, and transactions
- Reading the blockchain proceeds backward by following hashes from newest to oldest blocks

Merkle Tree Protection Layer

Purpose

Ensures transaction integrity within blocks and detects any tampering immediately.

Structure

- Each transaction is hashed individually (H1, H2, H3, etc.)
- Pairs of hashes are combined and hashed again to create parent nodes
- Process continues recursively until reaching the Merkle root
- Both actual transactions and Merkle root are stored in the block header

Tampering Detection

- If any transaction is modified, its hash changes
- This cascades upward, changing the Merkle root
- Any modification is immediately detectable because the root will not match the recalculated value

Key Differences: Blockchain vs. Linked Lists

Aspect	Linked List	Blockchain
Connection Method	Memory address pointers	Cryptographic hashes
Reading Direction	Forward from head node	Backward from newest block
Storage Location	Single computer	Multiple nodes in a network
Data Type	Flexible (nodes)	Transactions
Block	Can be	Always

Addition	added anywhere	added at the end

Decentralization and Tamper-Proofing

How Blockchain Achieves Tamper-Proofing

- Multiple nodes store identical copies of all blocks
- Nodes compare their block copies with each other
- If one block is modified, its hash changes, breaking the chain
- The modified block is identified and compared against copies in other nodes
- The majority copy (typically 51%+) is considered authentic
- Modified blocks are discarded, maintaining integrity across the network

Consensus Mechanism

- Protocol used by blockchain nodes to agree on the authentic version of blocks
- Ensures all nodes maintain identical copies of valid blocks
- Enables nodes to validate and share transactions simultaneously

Public vs. Permissioned Blockchains

- **Permissioned Blockchain**: Specific nodes designated by an organization (e.g., enterprise blockchains)
- **Public Blockchain**: Any node can join by following network protocols (e.g., Bitcoin)
  - May require proof of work (solving computational problems)
  - May require proof of stake (providing storage or processing power)

Tree and Graph Application in Blockchain

- Trees: Used in the form of Merkle trees to verify transaction integrity
- Graphs: Graph traversal algorithms determine if a node is part of the blockchain network and validate network connections

Blockchain Node Functions

A blockchain node must:

- Follow the established blockchain protocol
- Connect to other nodes in the network
- Use cryptographic keys and hashes for identity and validation
- Validate incoming blocks
- Store copies of valid blocks
- Broadcast valid blocks and transactions to

other nodes simultaneously

Common Blockchain Applications

Cryptocurrencies

- Most well-known application (Bitcoin, Ethereum, etc.)
- Technically tokens rather than currency (not issued by central banks)
- Community agreement establishes perceived value
- Decentralized ledger prevents tampering

Smart Contracts

- Self-executing contracts with conditions built into code
- Automatically execute when specified conditions are met
- Tamper-proof storage on blockchain

eliminates need for extensive legal oversight
- Reduces dependency on lawyers and legal fees
- Example: Construction contracts with automatic penalty clauses

Emerging Applications

- Revenue Records: Land registries and property records
- Health Records: Electronic health records and medical histories
- Supply Chain: Tracking product origin and authenticity
- Voting Systems: Secure, tamper-proof voting records

Implementation Challenges

Despite blockchain's robustness, widespread adoption faces barriers:

- High Implementation Costs: Expensive infrastructure and maintenance requirements
- Return on Investment: Benefits must justify significant financial investment
- Limited Scalability: Current systems struggle with transaction volume
- Regulatory Uncertainty: Legal frameworks still developing

Current adoption is limited to cryptocurrency and smart contracts, with pilot programs in revenue and health record management. Widespread implementation will increase as costs decrease and return on investment improves.

Key Takeaways

1. Blockchain combines linked list structure with cryptographic hashing for secure data

management
2. Unlike linked lists, blockchain is fundamentally a decentralized network system
3. Hash functions ensure block integrity, while Merkle trees verify transaction integrity within blocks
4. Multiple distributed copies of blocks make blockchain practically tamper-proof
5. Consensus mechanisms enable nodes to maintain identical, authentic copies
6. While blockchain is secure, cost-effectiveness determines real-world adoption
7. Understanding blockchain requires knowledge of linked lists, hash functions, trees, and graphs

Lecture 21

Search Algorithms and Algorithmic Complexity

Overview

This module focuses on search algorithms and their complexity analysis. Search is a fundamental operation in computer science, present in everyday internet activities such as using search engines, shopping platforms, booking systems, and streaming services. Before diving into specific search algorithms in subsequent modules, understanding algorithmic complexity is essential to determine which approach is most efficient for a given problem.

Why Search Algorithms Matter

- Practical Relevance: Every internet activity involves search in some form—from web searches to product recommendations to

train bookings

- **Efficiency Concerns**: Users expect immediate results; delays lead to poor user experience and loss of customers
- **Business Perspective**: Companies must evaluate solutions based on feasibility, cost (computing resources), speed of delivery, and scalability

Algorithmic Complexity Fundamentals

Definition and Purpose

Algorithmic complexity describes how much time an algorithm requires as the input size increases. The focus is on the **growth rate** rather than exact execution time in seconds or hours.

Key Point: We are concerned with scalability—if input size increases from 100 to 10 lakh elements, how does the algorithm's performance

change?

Why Complexity Matters

- Different algorithms can solve the same problem but require vastly different resources
- Time complexity is prioritized over space complexity because faster responses improve user satisfaction
- Managers and programmers need to understand algorithmic efficiency for system evaluation

Types of Complexity Analysis

When analyzing algorithms, three scenarios are considered:

- **Best Case**: The situation where the algorithm finds the solution very quickly with minimal

work
- Worst Case: The situation requiring maximum effort to find the solution
- Average Case: The typical amount of work required, falling between best and worst cases

Example: In a numbered housing society with 24 houses accessed through gate A, houses 3 and 4 represent the best case (closest to gate), houses 19 and 24 represent the worst case (farthest from gate), and houses 7-18 represent the average case.

Big O Notation

Big O notation describes the order of magnitude of how algorithmic work grows with input size. It answers the question: "If I increase the input size, in what proportion does the time requirement increase?"

Common Complexity Classes (from fastest to slowest)

Notation	Name	Description	Growth Pattern
O(1)	Constant	Irrespective of input size, effort remains the same	Flat line
O(log n)	Logarithmic	Work increases slowly; typical of divide-and-	Slow growth

		conquer algorithms	
O(n)	Linear	Input size and time increase proportionally; if input doubles, time doubles	Straight line
O(n log n)	Linearithmic	Slightly faster growth than linear; faster	Moderate growth

		than O(n) but slower than O(log n)	
$O(n^2)$	Quadratic	Work increases dramatically; when input is 2, effort is 4; when input is 5, effort is 25	Rapid growth
$O(2^n)$	Exponent	Extremely	Very

	ial	rapid growth; typically impractical for large inputs	rapid growth

Understanding Fast vs. Slow Algorithms

- Fast algorithm = Low complexity = Slower growth rate (O(1), O(log n), O(n))
- Slow algorithm = High complexity = Faster growth rate ($O(n^2)$, $O(2^n)$, $O(n^3)$)

Practical Analogy: The chess legend illustrates exponential growth—doubling grains on each square of a chessboard results in astronomically large numbers that quickly become unmanageable.

Complexity of Data Structure Operations

Arrays

- **Access by Index**: O(1) — Direct access to known position
- **Search (linear)**: O(n) — Must check elements sequentially if position unknown
- **Insert at End**: O(1) — Always appends without moving other elements
- **Insert at Beginning**: O(n) — Requires shifting all elements
- **Insert in Middle**: O(n) — Requires shifting subsequent elements
- **Update by Index**: O(1) — Direct access to known position
- **Update by Search**: O(n) — Must search first
- **Delete by Index**: O(1) — Direct removal from

known position
- Delete by Search: O(n) — Must search first

Linked Lists

- Insert at End: O(1) — Simply connect new node to last node's pointer
- Insert at Beginning: O(n) — May require search
- Insert in Middle: O(n) — Must search for insertion point

Stacks

- Push: O(1) — Always adds at top; location is fixed
- Pop: O(1) — Always removes from top; location is fixed
- Search: O(n) — Must pop elements until finding target

Queues

- **Enqueue**: O(1) — Always adds at end; location is fixed
- **Dequeue**: O(1) — Always removes from front; location is fixed
- **Search**: O(n) — Must dequeue elements until finding target

Trees

- **General Tree Traversal/Search**: O(n) — Must visit every node; increases proportionally with tree size
- **Binary Search Tree Search**: O(log n) — At each level, half the remaining nodes are eliminated by choosing one branch
  - **Reason**: Left subtree contains smaller values, right subtree contains larger values; this property allows subdivision of

the problem into one relevant sub-problem

Graphs

- Traversal/Search (BFS or DFS): O(V + E) — Must potentially visit all vertices and edges
  - V = number of vertices
  - E = number of edges

Practical Recommendations

Business Preference Order:

1. O(1) — Ideal but rarely achievable in real scenarios
2. O(log n) — Excellent; highly scalable
3. O(n log n) — Good; manageable for most applications
4. O(n) — Acceptable; straightforward scaling
5. $O(n^2)$, $O(2^n)$, $O(n^3)$ — Problematic; avoid unless necessary

Handling Highly Complex Algorithms

When algorithmic complexity becomes $O(n^2)$ or exponential, businesses often employ **heuristics**—rules of thumb that provide good enough (though not optimal) solutions rather than perfect answers. This pragmatic approach balances computational feasibility with practical results.

Key Takeaways

- Algorithmic complexity measures growth rate, not absolute time
- Big O notation enables comparison of algorithms across different input sizes
- Scalability is crucial—how does performance change as data volume increases?
- Choice of algorithm directly impacts system responsiveness and user experience

- O(1) and O(log n) complexities are preferable; exponential complexities should be avoided
- Understanding data structure operations and their complexities is fundamental to choosing appropriate solutions

Lecture 22

Search Algorithms Overview

This lecture continues the module on search algorithms, building on previous discussions about algorithm complexity and search operations across different data structures. The session covers five key search strategies: linear search, binary search, jump search, exponential search, and hash-based search.

Linear Search

- Definition: Searches sequentially through elements one by one, starting from the first element
- Requirement: Works on both sorted and unsorted data
- Process: Compare each element with the search term; move to the next element if no match
- Time Complexity: O(n) - effort increases proportionally with data size
- Application: Works on arrays, linked lists, trees, and graphs
- Limitation: Becomes time-consuming with large datasets (e.g., searching millions of records)

Binary Search

- Definition: Algorithm that divides a sorted list into halves to locate elements

- **Requirement**: Data must be sorted in increasing order
- **Process**:
  - Start at the middle element
  - Compare with search term
  - If smaller, search the right half; if larger, search the left half
  - Repeat with the middle of the remaining segment
- **Time Complexity**: O(log n) - divides the problem into smaller sub-problems
- **Key Difference from Binary Search Tree**: Binary search is an algorithm applicable to any sorted list, while a Binary Search Tree (BST) is a data structure that inherently maintains order through its structure

Binary Search Tree (BST) vs. Binary Search

Aspect	Binary Search	Binary Search Tree
Definition	Algorithm	Data structure
Requirement	Sorted list needed	Maintains order through structure
Complexity (balanced)	O(log n)	O(log n)
Complexity (unbalanced)	N/A	O(n) - behaves like a linked list
Application	Wider applicability	Specific tree structure

Jump Search

- **Definition**: Algorithm that jumps by a fixed number of positions rather than checking sequentially
- **Requirement**: Data must be sorted
- **Jump Size Calculation**: Determined by √n (square root of data size)
  - Example: For 100 elements, jump size is 10; for 9 elements, jump size is 3
- **Process**:
  - Jump by fixed intervals until the search term is exceeded
  - Return to the previous position
  - Perform linear search forward from that point
- **Time Complexity**: O(√n) - divides data into √n blocks
- **Advantage**: Divides problem into

manageable sub-problems without the large jumps of binary search

Exponential Search

- Definition: Algorithm that exponentially increases jump size until the search term is found
- Requirement: Data must be sorted
- Jump Pattern: Starts with jump of 1, then doubles successively (1, 2, 4, 8, 16...)
- Process:
  - Make exponentially larger jumps until the search term is exceeded
  - Once the range is identified, perform binary search within that block
- Key Difference from Jump Search:
  - Jump search uses fixed-size jumps with linear search within blocks
  - Exponential search uses increasing jumps

with binary search within blocks

- Time Complexity: O(log n) - combines exponential jumps with binary search
- Advantage: Efficient for very large and continuously growing databases

Hash-Based Search

- Definition: Uses a hash function to calculate an index location for data storage and retrieval
- Hash Function: Converts input values into unique indices (e.g., mod 10 returns 0-9)
- Collision Handling: Uses chaining method where multiple values pointing to the same index are linked like a linked list
- Process:
  - Calculate hash value of the search term
  - Go directly to that index
  - If collision exists, search through the

chain linearly

- Time Complexity:
  - Average case: O(1) - fixed number of steps regardless of database size
  - Worst case: O(n) - occurs when all data points to the same index (collision intensive)
  - Balanced chain scenario: O(log n) - if chains are organized as balanced trees
- Advantage: Fastest search method when hash function distributes data evenly
- Note: Does not require sorted data

Use Case Analysis

Use Case	Recommended Algorithm	Reason
Find a guest in today's	Linear Search	No inherent order;

hotel check-in register		unstructured data
Find a product in a sorted catalog by categories	Jump Search	Sorting by categories (non-numeric); fixed jumps work better
Find student record using roll number in database	Hash Search	Databases use indexes; hash provides O(1) average access
Find a product in catalog sorted by	Binary Search	Numeric sorting allows value comparison

price		
Search financial transactions in rapidly growing system	Exponential Search	Unknown growth; exponential jumps handle scaling efficiently

Key Takeaways

- Linear Search: One-by-one sequential search; works anywhere but slow for large datasets
- Binary Search: O(log n) complexity; requires sorted list and divides into two halves
- Jump Search: O(√n) complexity; balances number of jumps with block size
- Exponential Search: O(log n) complexity; exponentially increases jumps then applies

binary search
- Hash Search: O(1) average complexity; calculates exact location; fastest for evenly distributed data
- Selection Factors: Algorithm choice depends on data size, sorting status, data distribution, and growth rate

Lecture 23

Tree Types, Hashing, and Blockchain: Concluding Non-Linear Data Structures

Course Announcement and Quiz Information

- Quiz Two scheduled for October 31st with timing to be communicated separately
- Best of two quizzes will count toward final

marks—whichever quiz score is higher will be used
- Attendance in at least one quiz is mandatory; students who missed Quiz One should definitely attend Quiz Two
- Syllabus for Quiz Two: Complete non-linear data structures including blockchain (does not include linear data structure content from Quiz One)
- Doubt clearance session scheduled for the week before Quiz Two
- Non-linear data structure section concludes with this lecture

---

Tree Types

General Tree

- Definition: A tree where the number of children per node is not fixed; each node can have zero, one, two, or many children
- Key Property: Each child has exactly one parent
- Real-World Applications:
  - Folder structures (directories and subdirectories in file systems)
  - Family trees
  - Organization hierarchies (executives, managers, employees)
  - Biological taxonomy (kingdom, phylum, class, order, etc.)

Binary Tree

- Definition: A tree where each node has at most two children (left and right child)

- Key Property: The number of children can be 0, 1, or 2
- Applications:
  - Tournament structures (tennis, badminton—individual sports with knockout rounds)
  - Decision trees and flowcharts (yes/no branching)
  - Expression evaluation

Complete Binary Tree

- Definition: A binary tree where all levels are completely filled except possibly the last level, which is filled from left to right
- Constraint: All nodes except possibly the last level have exactly two children
- Characteristic: If a node exists on the last level, all positions to its left must be filled

Binary Search Tree (BST)

- Definition: A binary tree with an additional property where left child value < parent value and right child value ≥ parent value
- Key Characteristics:
  - Enables efficient searching similar to searching in a physical dictionary
  - If values are inserted in sorted order, the tree can become skewed (like a linked list), reducing search efficiency
  - Used for quick lookup operations

AVL Tree

- Purpose: Solves the skewness problem of Binary Search Trees
- Key Property: Maintains balance by ensuring the height difference between left and right subtrees is at most 1
- Mechanism: Uses rotations to rebalance the tree when height difference exceeds one

level
- **Benefit**: Guarantees efficient search regardless of insertion order
- **Applications**: Operating system memory indexing, library catalog systems

Heap

- **Definition**: A specialized binary tree where the parent-child relationship follows a specific ordering
- **Types**:
  - **Max Heap**: Parent node value is always greater than child node values (root contains maximum)
  - **Min Heap**: Parent node value is always less than child node values (root contains minimum)
- **Note**: No left-right ordering requirement, only parent-child relationship matters

- Application: Priority queues (higher priority items processed first; implementation depends on whether priority 1 is highest or priority 10 is highest)

Hashing

Basic Concept

- Hash Function: Takes an input of any size and produces a fixed-length output
- Key Property: The output length remains constant regardless of input size
- Simple Example: Using modulo (mod) operator to find remainder when dividing by a number

Modulo Function

- **Definition**: A basic hash function that divides input by a number and returns the remainder
- **Example**:
  - 101 mod 10 = 1 (last digit)
  - 27 mod 5 = 2 (27 ÷ 5 = 5 remainder 2)
- **Possible Values**: For mod n, possible outputs are 0 to (n-1)
- **Use Case**: Distributing data into buckets; helps organize large datasets

Hash Table (Key-Value Pairs)

- **Structure**: Stores data as key-value pairs with indexed buckets
- **Process**:
  A. Calculate hash of input
  B. Use hash value to determine bucket location

C. Store or retrieve data from that bucket

- **Advantage**: Enables faster searching through indexing rather than sequential search
- **Application**: Database indexing for efficient data retrieval on billions of records

Hash Collision

- **Definition**: Occurs when two different inputs produce the same hash output
- **Example**: Numbers ending in 2 (12, 22, 32) all have hash value 2 using mod 10
- **Solution - Chaining**: Link items with the same hash value using linked lists, pointing subsequent items to the previous item at that index
- **Trade-off**: Chaining increases search time slightly but keeps most data in correct bucket

Cryptographic Hash Properties

- **Deterministic**: Same input always produces same output
- **Fixed Length**: Output length is constant regardless of input size
- **One-Way Function**: Cannot reverse the process to obtain original input from hash (critical for security)
- **Avalanche Effect**: Single character change in input produces completely different hash output
- **Collision Resistance**: Extremely difficult (practically impossible) to find two different inputs with same hash

Applications of Hashing

- **Password Storage**: Databases store password hashes rather than actual

passwords; when user logs in, entered password is hashed and compared to stored hash
- Integrity Checks: Calculate hash of original data/file, transmit both separately; recipient recalculates hash and compares to verify no tampering occurred
- Indexing: Database systems use hash functions to create indexes for quick data retrieval

---

Blockchain

Core Concept

- Definition: A data structure consisting of blocks linked in a chain, where each block

contains the hash of the previous block
- Key Characteristic: A block stores its own transaction data, its own hash, and the previous block's hash

Block Structure

- Contents:
  - Set of transactions (banking records, health reports, university records, etc.)
  - Hash of current block (calculated from current transactions)
  - Hash of previous block (creates the chain link)
- Genesis Block: First block has no previous block; its previous hash field is null or zero

Why Blockchain is Secure

Immutability through Hashing:

- If any transaction in a block is changed, that block's hash changes completely
- This causes mismatch with the next block's "previous hash" reference
- The mismatch propagates through all subsequent blocks, making tampering obvious

Decentralization:

- Blockchain stores copies of all blocks across multiple nodes in a network
- All nodes maintain identical copies of the complete ledger
- Single node tampering is detected by comparing with other nodes

Consensus Mechanism:

- Network nodes must agree before accepting new blocks

- Makes simultaneous hacking of 51% of nodes impractical (theoretically possible but resource-intensive)

Merkle Tree

- Definition: A tree structure using hashing for integrity verification of multiple transactions within a block
- Process:
  A. Hash each individual transaction (T1, T2, T3, T4, etc.)
  B. Pair hashes and hash their combination (creating intermediate nodes)
  C. Continue combining and hashing until reaching single root hash
- Purpose: Creates layered security; any transaction change alters the root hash, signaling tampering
- Advantage: Provides transaction-level

integrity verification

Types of Blockchain

Public Blockchain:

- Anyone with processing power can join the network
- Most popular example: Bitcoin
- Uses Proof of Work: nodes solve complex computational problems (mining) to validate blocks
- Mining difficulty increases over time as more nodes join
- Highly resource-intensive but completely decentralized

Enterprise Blockchain:

- Private blockchains used by organizations across multiple locations
- Fewer nodes than public blockchains

- More cost-effective for specific organizational needs
- IBM has offered enterprise blockchain solutions
- Generally not adopted widely due to costs

Applications of Blockchain

Cryptocurrency:

- Bitcoin is the most well-known application
- All transactions are recorded permanently with complete traceability
- New blocks continuously added; no deletion allowed
- Tamper-proof due to hash linking and decentralization

Smart Contracts:

- Blockchain-based agreements that execute automatically

- More secure than paper-based contracts as changes are recorded permanently
- Contract terms cannot be altered without detection

Record Management:

- Government records (revenue reports, property records)
- Health records
- University transcripts and academic records
- State pilot programs: Andhra Pradesh, Telangana, and Estonia have tested blockchain for various records
- Limited adoption due to high costs

Blockchain vs. Linked List

- Linked List: Uses memory addresses to point to next node; data typically stored on single system

- Blockchain: Uses hash values to link blocks; data stored across multiple nodes (decentralized)
- Both chain data but differ fundamentally in linking mechanism and distribution

Security Consideration

- 51% Attack: Theoretically possible if attacker controls 51% of network nodes simultaneously, but practically infeasible due to resource requirements and simultaneous coordination needed
- Current Limitation: High storage requirements as all data is replicated across nodes, making blockchain expensive for implementation

---

Conclusion

Non-linear data structures have been comprehensively covered through tree types, hashing mechanisms, and blockchain applications, demonstrating how these concepts work together to create secure, efficient data management systems.

Lecture 24

Network Search Algorithms: Overview and Implementation

This lecture concludes Module 4 on search algorithms by exploring search within nonlinear data structures, specifically networks represented as graphs and trees.

Introduction to Network Search

- Network search extends search concepts beyond linear data structures to include trees and graphs
- A network consists of nodes (vertices) and edges connecting them
- Search in networks often involves finding paths between nodes rather than just locating individual elements

Traversal Methods as Search Techniques

Tree Traversals

- Pre-order traversal: Root, left, right (reads data as traversal occurs)
- In-order traversal: Left, root, right
- Post-order traversal: Left, right, root
- Traversals inherently perform search by reading node data during the process

Level-Order Traversal (Breadth-First Search)

- Moves level by level from top to bottom, left to right
- Useful for understanding network hierarchy and levels
- Implemented using a queue data structure
- Helps identify how many hops are needed to reach different nodes

Graph Search Algorithms

Depth-First Search (DFS)

- Traverses as far as possible along each branch before backtracking
- Useful for understanding complete network topology, especially when identifying new nodes or leaves
- Implemented using a stack data structure

- Effective for mapping network structure when new systems or connections are added

Breadth-First Search (BFS)

- Traverses level by level based on hop count rather than geographic location
- Helps understand hierarchical network topology (e.g., client-based systems, intermediate servers, main servers)
- Useful for identifying levels within a network structure

Shortest Path Problem

Real-World Applications

- Simply knowing network topology is insufficient for many practical applications
- Once familiar with a network, users need targeted navigation rather than complete

exploration
- Shortest path algorithms help find optimal routes between specific source and destination nodes

Defining "Shortest"

- Shortest does not necessarily mean the shortest distance
- Cost variables can include:
  - Distance (in kilometers)
  - Time taken
  - Monetary cost (toll charges)
  - A combination of multiple factors
- Path selection depends on specific requirements (e.g., avoiding toll roads, faster travel time despite longer distance)

Dijkstra's Algorithm

Overview

- Named after Dutch computer scientist Edsger Dijkstra (pronounced "DIK-struh")
- Finds the shortest path from a source node to all other nodes in a weighted graph
- Works only with **positive weights**—negative edge weights are not permitted

Basic Assumptions

- Each edge has an associated cost (weight)
- All weights must be non-negative
- Cost represents the variable being optimized (distance, time, toll charges, etc.)

Algorithm Steps

1. **Initialize**: Set source node distance to 0; set all other nodes to infinity

2. Select unvisited node: Choose the unvisited node with the smallest known distance
3. Update distances: For each neighbor, check if there is a shorter path through the current node
4. Mark as visited: Once a node's shortest distance is finalized, mark it as visited
5. Repeat: Continue until all nodes are visited or the destination is reached

Worked Example

- Starting from node A with weighted edges:
  - Direct path A→B: cost 4
  - Direct path A→C: cost 2
  - Path A→C→B: cost 2+1=3 (shorter than direct path)
  - Path A→C→D: cost 2+5=7
  - Path A→C→B→D: cost 3+3=6 (shorter than previous)

    - Shortest path to each node is finalized once visited

Key Characteristics

- Calculates shortest distances from one specific starting node to all other nodes
- Does not provide absolute shortest distances between all node pairs
- Results vary depending on which node is selected as the starting point

Practical Applications

Spoke and Hub Model

- Used by courier companies and logistics firms to optimize delivery networks
- Instead of connecting every city to every other city, businesses create hubs with spoke cities

- Example: Package from Jodhpur to Somnath travels Jodhpur→Jaipur→Delhi→Somnath
- Dijkstra's algorithm calculates shortest distances between major hubs
- Reduces overall network complexity and cost

Current Use Cases

- GPS navigation and route planning
- Food delivery services (Zomato, Swiggy) for driver allocation
- Telecommunications network optimization
- Courier and logistics networks
- Computer network routing

Alternative Network Search Algorithms

A* Algorithm

- Combines actual distance cost with heuristic guidance (sense of direction)

- Determines if travel is in the correct direction (e.g., traveling south instead of north)
- More optimal than Dijkstra's algorithm
- Widely used in GPS navigation systems

Other Variants

- **Uniform Cost Search**: Explores lowest-cost paths; useful for avoiding tolls or specific constraints
- **Bidirectional Search**: Searches simultaneously from source and destination to find meeting point
- **Floyd-Warshall Algorithm**: Finds shortest paths between all pairs of nodes
- **Bellman-Ford Algorithm**: Works with both positive and negative edge weights
- **Prim's Algorithm**: Connects all nodes with minimum total connection cost (minimum spanning tree)

- Kruskal's Algorithm: Creates minimum-cost connected network; alternative to Prim's algorithm

Algorithm Selection Criteria

- Dijkstra's Algorithm: When shortest path needed from one source to all destinations
- Bellman-Ford: When negative edge weights exist
- Prim's/Kruskal's: When connecting all nodes with minimum total cost
- Floyd-Warshall: When all-pairs shortest paths are required
- A*: When directional heuristics can optimize search

Key Takeaways

- Network search algorithms focus on finding

optimal paths between nodes in weighted graphs

- Dijkstra's algorithm is a fundamental approach for single-source shortest path problems
- Works on weighted graphs with non-negative weights
- Edge weights represent various costs: distance, time, toll charges, or combinations thereof
- Different algorithms serve different business scenarios and network requirements
- Most real-world applications already have implemented variants or combinations of these algorithms
- Understanding algorithm selection is more important than implementing from scratch in practical scenarios

Lecture 25

Introduction to Sorting Algorithms and Bubble Sort

Overview

This lecture introduces the final module on sorting algorithms, focusing on understanding what sorting is and how the bubble sort algorithm works. Sorting is fundamental in computer science, essential for both data organization and enabling efficient searching.

What is Sorting?

- Definition: Sorting is the process of arranging elements in a specific order (ascending, descending, or based on specific parameters)
- Parameters: Elements can be sorted by

various criteria such as numerical values, alphabetical order, relevance, price, quality, or any custom parameter

- Connection to Searching: Most search algorithms assume data is already sorted, making sorting a prerequisite for efficient searching
- Real-World Applications:
  - E-commerce platforms sort products by relevance, price, quality, and other metrics
  - Flight booking systems sort results by price, travel time, and other criteria
  - Academic reporting requires sorting marks from highest to lowest for grade assignment
  - Multiple sorting parameters enhance user experience and decision-making

Bubble Sort Algorithm

Basic Concept

- The name derives from the behavior of air bubbles rising in water
- Core Idea: Elements gradually move to their correct positions through successive comparisons and swaps
- Lighter elements (smaller values) move toward the beginning; heavier elements (larger values) move toward the end

Classic Method (Largest Element to the End)

- Start from the first element and compare it with the next element
- If the first element is larger, swap their positions
- Move to the next pair and repeat
- After each pass (complete iteration through the list), the largest unsorted element moves

to its correct position at the end
- Repeat the process, requiring one fewer comparison in each subsequent pass
- Total passes needed equals the number of elements in the list

Step-by-Step Example

Using the list: [10, 5, 8, 4, 3, 2]

Pass 1: Compare and swap adjacent elements

- 10 > 5 → swap → [5, 10, 8, 4, 3, 2]
- 10 > 8 → swap → [5, 8, 10, 4, 3, 2]
- 10 > 4 → swap → [5, 8, 4, 10, 3, 2]
- 10 > 3 → swap → [5, 8, 4, 3, 10, 2]
- 10 > 2 → swap → [5, 8, 4, 3, 2, 10]
- Result: Largest element (10) is now in correct position

Pass 2 onward: Repeat with reduced comparisons since last element is already sorted

- After Pass 2: Two largest elements in correct positions
- Continue until entire list is sorted

Alternative Method (Smallest Element to the Front)

- Start from the last element and compare with the previous element
- Focus on finding the smallest element and moving it left
- After each pass, the smallest unsorted element moves to its correct position at the beginning
- This variant produces the same final result through a different approach

Sorting Non-Numerical Data

Bubble sort works on any comparable data

(alphabetical, strings, etc.)

- Example with names: [Mohan, Ram, Sham, Lakshman, Arjun, Bheem]
- Capital letters are compared alphabetically (A < Z)
- Same comparison and swapping logic applies

Time Complexity Analysis

Calculation

- Pass 1: n comparisons (where n is the number of elements)
- Pass 2: n - 1 comparisons
- Pass 3: n - 2 comparisons
- Final pass: 1 comparison

Total comparisons = $n + (n-1) + (n-2) + \ldots + 1 = n(n-1)/2 = \frac{1}{2}n^2 - \frac{1}{2}n$

Big O Notation

- Using Big O rules:
  - Discard lower-order terms: -½n is negligible compared to n^2
  - Remove numerical constants: ½ is removed
- Result: $O(n^2)$ - Quadratic complexity

Impact on Performance

- Small lists (5-6 elements): Manageable with ~25 operations
- Larger datasets: Complexity grows rapidly (10,000 elements = ~100 million comparisons)
- Conclusion: Unsuitable for large datasets or business applications despite logical simplicity

Key Takeaways

1. Sorting arranges data in a specific order based on parameters (numerical, alphabetical, relevance, etc.)
2. Bubble sort uses successive comparisons of adjacent elements with swaps to order data
3. Each pass moves one element to its correct final position
4. Multiple passes required equal to the list size
5. Time complexity is $O(n^2)$, making it impractical for large datasets
6. Suitable only for small lists despite being conceptually simple
7. Next algorithms will address efficiency concerns for larger data sets

Lecture 26

Selection Sort: A Comprehensive Overview

This lecture covers **selection sort**, one of the fundamental sorting algorithms that builds on the concepts previously learned from bubble sort. Selection sort operates by repeatedly finding the smallest (or largest) element from the unsorted portion of a list and placing it in its correct position.

Core Concept and Mechanism

- **Basic Principle**: Selection sort selects one element at a time and places it in its correct position through swapping
- **Classical Implementation**: Finds the minimum element from the unsorted portion and swaps it to the front
- **Process Flow**:

- Start at the first position and assume it contains the smallest element
- Compare this element with all remaining elements to the right
- When a smaller element is found, mark it as the new minimum
- After traversing the entire unsorted portion, swap the minimum with the starting position
- Move to the next position and repeat the process

Execution with Numerical Example

Using the array [10, 5, 8, 4, 3, 2]:

- Pass 1: Identify 2 as the minimum among all elements, swap it with 10 → [2, 5, 8, 4, 3, 10]
- Pass 2: Identify 3 as the minimum from position 2 onward, swap with 5 → [2, 3, 8, 4, 5, 10]

- Pass 3: Identify 4 as the minimum from position 3 onward, swap with 8 → [2, 3, 4, 8, 5, 10]
- Pass 4: Identify 5 as the minimum from position 4 onward, swap with 8 → [2, 3, 4, 5, 8, 10]
- Final Result: Array is sorted after n-1 passes

Alphabetical Sorting Application

Selection sort can be applied to alphabetical data where letters are compared lexicographically (A < B < C ... < Z). The algorithm works identically—finding the minimum (earliest) letter and placing it in correct position before moving to the next unsorted portion.

Variants of Selection Sort

Minimum Selection Sort (Classical):

- Searches for the smallest element starting from the beginning
- Places minimum elements at the front of the list progressively

Maximum Selection Sort:

- Searches for the largest element starting from the end
- Begins from the last position and works backward
- Places maximum elements at the end of the list

Bidirectional Selection Sort:

- Performs both minimum and maximum search in a single pass
- Finds minimum from the front and maximum from the back simultaneously
- Places both elements in correct positions in

one pass
- Reduces number of passes by half compared to unidirectional variants

Time Complexity Analysis

- **All Variants**: $O(n^2)$ complexity
- **Why Bidirectional Doesn't Improve Complexity**: Although passes are reduced by half, the total number of comparisons remains the same because comparisons still need to traverse from both ends to the middle
- **Practical Impact**: For larger lists, $O(n^2)$ complexity makes selection sort inefficient despite its simplicity

Comparison with Bubble Sort

- **Similarity**: Both are $O(n^2)$ algorithms

- Key Difference in Swaps:
  - Bubble sort may perform multiple consecutive swaps for the same element in a single pass (up to n-1 swaps)
  - Selection sort performs only one swap per pass after finding the minimum/maximum
- Advantage: Selection sort is more efficient in terms of the number of swaps performed

Key Takeaways

- Selection sort is fundamentally based on selecting one element per pass and placing it in its correct position
- The algorithm uses swapping to achieve sorting with minimal operations per iteration
- Despite having multiple variants (minimum, maximum, bidirectional), all maintain $O(n^2)$ time complexity
- Selection sort is suitable for educational

purposes and small datasets but should be avoided for larger lists due to quadratic complexity
- While better than bubble sort in terms of swap operations, it remains impractical for real-world applications with large datasets

Lecture 27

Insertion Sort: Concept, Implementation, and Use Cases

Overview

Insertion sort is the third sorting algorithm in the module, following bubble and selection sort. Unlike previous algorithms that use swapping, insertion sort works by moving elements to create space and inserting items at their correct positions. The algorithm processes the list by

treating the left portion as sorted and progressively inserting each subsequent element into its correct position within that sorted portion.

Core Concept and Mechanism

- Key Principle: Insertion sort picks the next item from the unsorted portion and inserts it into its correct position within the already sorted portion
- Movement vs. Swapping: Unlike bubble and selection sort, insertion sort uses element movement rather than swapping. Elements shift one or more positions to the right or left to make space for the key element
- Starting Point: The algorithm begins with the second item from the beginning of the list, assuming the first element is already sorted

Algorithm Process (Minimum/ Classical Insertion Sort)

- Step 1: Start with the second item as the key and compare it with items to its left
- Step 2: Move any element larger than the key one position to the right
- Step 3: Continue comparing until finding an element smaller than the key or reaching the list's beginning
- Step 4: Insert the key at the vacant position
- Step 5: Repeat the process for each subsequent element until the entire list is sorted

Numeric Example Walkthrough

When applied to the list [10, 5, 8, 4, 3, 2]:

- Pass 1: Key is 5; 10 moves right; 5 is

inserted → [5, 10, 8, 4, 3, 2]

- Pass 2: Key is 8; 10 moves right; 8 is inserted → [5, 8, 10, 4, 3, 2]
- Pass 3: Key is 4; all larger elements (10, 8, 5) move right; 4 is inserted → [4, 5, 8, 10, 3, 2]
- Pass 4: Key is 3; all larger elements move right; 3 is inserted → [3, 4, 5, 8, 10, 2]
- Pass 5: Key is 2; all elements move right; 2 is inserted → [2, 3, 4, 5, 8, 10]

Alphabetical Example Application

When sorting names alphabetically (Mohan, Ram, Sham, Lakshman, Arjun, Bheem, Nakul):

- Process starts from the second name (Ram) and compares with items to its left
- Each name is inserted in its alphabetically correct position by moving larger names to the right
- Result follows alphabetical order with

movements rather than swaps

Variants of Insertion Sort

Minimum Insertion Sort (Classical)

- Finds and places minimum elements at the beginning
- Larger elements move to the right
- Starts from the beginning of the list

Maximum Insertion Sort

- Finds and places maximum elements at the end
- Smaller elements move to the left
- Starts from the second item from the end
- Operations are opposite to minimum insertion sort

Bi-directional Insertion Sort

- Combines minimum and maximum approaches
- Starts from both beginning and end simultaneously
- Finds both minimum and maximum elements in a single pass
- Places them at their correct respective positions

Time Complexity

- Complexity Order: O(n²) for all variants
- Complexity increases by the square when list size increases (4→9, 5→16, 6→25, etc.)
- Same complexity for all variants regardless of starting direction, making the classical implementation sufficient for most purposes

Advantages and Disadvantages

Disadvantages:

- Inefficient for large, randomly ordered lists due to $O(n^2)$ complexity
- Requires multiple comparisons and movements for unsorted data

Advantages:

- Efficient for small lists
- Excellent for partially sorted or nearly sorted lists
- Significantly useful for inserting new elements into already sorted lists

Practical Use Cases

Highly Effective Scenarios:

1. New Item Insertion in Sorted List: When a new element arrives and needs to be added to an already sorted list (e.g., new student in

an alphabetically ordered class roster), insertion sort quickly finds the correct position without requiring full re-sorting

2. E-commerce Applications: Useful when inventory or product lists are already sorted by primary classification (e.g., by category) and new items need to be inserted in their correct subcategory positions
3. Partially Sorted Data: Performs well when the majority of the list is already sorted with only a few out-of-order elements
4. Dynamic Data Insertion: Ideal for scenarios where data arrives sequentially and needs to maintain sorted order in real-time

Summary

Insertion sort efficiently maintains sorted lists by inserting new elements into their correct positions through element movement rather

than swapping. While its $O(n^2)$ complexity makes it unsuitable for large random datasets, it excels in scenarios involving already-sorted lists receiving new insertions, making it particularly valuable in dynamic data management situations. The name "insertion sort" accurately reflects its fundamental operation: taking an element and inserting it into its proper place within the sorted portion of the list.

LECTURE 28

Overview of Search Algorithms and Complexity Analysis

Algorithm Complexity and Time Analysis

Big O Notation is used to measure how

algorithms scale with input size. Rather than calculating exact execution time, it analyzes the growth rate of processing time as input size increases.

Common Complexity Types (from fastest to slowest):

- O(1) - Constant time: Fixed number of operations regardless of input size (rare in practice)
- O(log n) - Logarithmic: Growth rate is relatively slow, ideal for practical applications
- O(n) - Linear: Operations increase proportionally with input size, manageable
- O(n log n) - Operations grow slightly faster than linear but still acceptable
- $O(n^2)$ - Quadratic: Rapid growth, impractical for large datasets
- $O(2^n)$ - Exponential: Extremely rapid growth, should be avoided

Array Operations Complexity

Operation	Complexity	Notes
Read with known index	O(1)	Direct access possible
Sequential read	O(n)	Must read elements one by one
Search	O(n)	Linear search through elements
Insert at end	O(1)	Single operation
Insert at	O(n)	Requires

beginning/ middle		shifting existing elements
Update at specific location	O(1)	Direct access and modification
Delete at specific location	O(1)	Single operation
Delete by searching	O(n)	Must search first, then delete

Linked List Operations Complexity

Operation	Complexity	Notes
Insert in	O(1)	Only requires

middle		updating links (2-4 operations)
Insert with search	O(n)	Search complexity dominates
Search	O(n)	Must traverse one by one

Stack and Queue Operations Complexity

- Push/Pop (Stack): O(1) - Fixed operations on top of stack
- Enqueue/Dequeue (Queue): O(1) - Fixed operations at front and rear
- Search in Stack/Queue: O(n) - Must remove

elements one by one to search

Tree Operations Complexity

Structure	Operation	Complexity
General Tree	Traversal/ Search	O(n)
Unbalanced Binary Tree	Search	O(n) - Becomes like a linked list
Balanced Binary Tree	Search	O(log n) - Reduces search space by half
Binary Search Tree (Balanced)	Search	O(log n) - Problem size halved at

Linear Search Algorithm

- Starts at first index and compares each element sequentially
- Complexity: O(n)
- Works on unsorted data
- Simple but inefficient for large datasets

Binary Search Algorithm

- Starts at middle index and divides search space in half based on comparison
- Complexity: O(log n)
- Requirement: Data must be sorted
- Much more efficient than linear search for large datasets
- Example: Searching 10 million records

requires approximately 24 comparisons

Jump Search Algorithm

- Makes fixed-size jumps through data (jump size = √n)
- When target is exceeded, moves back one block and performs linear search
- Complexity: O(√n)
- Less common but useful for scenarios where binary search isn't applicable
- Example: Finding a timestamp in a video by jumping in fixed intervals

Exponential Search Algorithm

- Jump size doubles with each step (1, 2, 4, 8, 16...)
- When target is exceeded, performs binary search on the final range

- Complexity: O(log n)
- Efficient when target location is unknown or data structure grows dynamically
- Better than jump search for certain scenarios

Hash Search Algorithm

- Uses hash function to calculate index position (e.g., modulo operation)
- Average case complexity: O(1)
- Worst case complexity: O(n) - When multiple elements hash to same index (collision)
- Requirement: Well-distributed hash function
- Practical complexity depends on hash function quality and data distribution

Graph Traversal Methods

Depth-First Search (DFS):

- Follows one path to its end before

backtracking
- Uses stack-based approach
- Complexity: O(V + E) where V = vertices, E = edges

Breadth-First Search (BFS):

- Explores level by level from source node
- Uses queue-based approach
- Complexity: O(V + E)

Dijkstra's Algorithm (Shortest Path)

Used to find shortest path between nodes in a weighted graph.

Key Requirements:

- Graph must be weighted (each edge has a cost/weight)
- Weights must be non-negative
- Works on directed and undirected graphs

Algorithm Process:

1. Set source node distance to 0, all others to infinity
2. Select unvisited node with smallest known distance
3. Update distances to neighboring nodes if shorter path found
4. Mark node as visited
5. Repeat until destination reached or all nodes visited

Example: Finding route from Home (A) to University (B):

- Direct path A→B might be 4 km
- Alternate path A→C→B might be 3 km (2 km + 1 km)
- Algorithm identifies the 3 km route as shortest

Complexity: $O(V^2)$ for basic implementation

Related Graph Algorithms

- *A Algorithm.** Combines actual cost with estimated distance, used in GPS navigation
- Floyd-Warshall: Finds shortest paths between all pairs of nodes
- Prim's/Kruskal's Algorithm: Finds minimum spanning tree for network connectivity

Practical Applications

Real-world use cases:

- Database queries with role-based search
- E-commerce product filtering and sorting
- GPS navigation and route planning
- Student record management and transcripts
- Food delivery route optimization